

Component Attribution in Agentic Knowledge Graph Question Answering: A Controlled Ablation, Budget Analysis, and Failure Census

Md. Sakif Khan^{a,*}, Sadia Sharmin^a

^a*Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh*

Abstract

Agentic systems for multi-hop question answering over knowledge graphs are assemblies of decomposition, guided search, backtracking, and self-correction that the literature reports as wholes. Which component produces a reported gain is rarely established. We build an agentic system, AGR, with four removable components, run it against four baselines on one frozen backbone under one call budget, and take it apart. On 400 questions from each of WebQSP and ComplexWebQuestions, AGR leads the agentic baseline by 9.5 and 8.7 Hits@1 points. The entire margin comes from questions on which the shared call budget truncates the baseline; where the baseline finishes, it is ahead. Removing the planner improves WebQSP F1 by 0.083 ($p = 0.006$) and cuts tokens by 31%. Three other components, a claim-verification layer among them, show no effect at the power available; we report the smallest effect each test could detect. A hand-read census of the residual failures finds label defects in 5.5% and 4.8% of the samples and names the *echo attractor*: a true, grounded entity one hop from the answer on which systems sharing a graph and backbone converge, so consensus-based label repair ratifies it.

Keywords: knowledge graph question answering, large language models, agentic retrieval, ablation study, component attribution, benchmark quality

*Corresponding author

Email address: sakifkhanopu@gmail.com (Md. Sakif Khan)

1. Introduction

Large language models answer factual questions fluently, and often wrongly. The errors are systematic rather than random, and a fabricated answer reads in the same register as a correct one, so they are hard to catch from the output alone [1]. The problem sharpens once an answer requires composing several facts. A wrong resolution at the first hop rarely announces itself downstream, and a correctness check on the final answer never sees the intermediate steps.

Retrieval-augmented generation [2] grounds the model in retrieved text, but it retrieves once, before reasoning begins, and the passage holding an intermediate entity need not resemble the question at all. A knowledge graph [3] turns composition into traversal. Static graph retrieval [4] still fetches a neighbourhood in one shot and cannot know what the second hop will need. Agentic systems close that gap by interleaving retrieval with reasoning: the model takes one step, observes what it found, and decides where to look next [5, 6, 7, 8]. Section 2 traces that line of work.

These systems are assemblies. A typical one decomposes the question into sub-goals, scores candidate edges with the model, backtracks from dead ends, and checks or rewrites its own output. The literature reports the assembly as a whole. When it outperforms a static baseline, three things could have produced the gain, and the report rarely separates them: a component, the budget regime the comparison ran under, or the labels of the benchmark itself. Later work then inherits the entire assembly on faith, along with its cost. The exception shows why the instrument matters. KBQA-o1’s own ablation removes its tree search and GrailQA F1 falls by thirty points [8]. That attribution exists because the authors took the system apart. This paper applies the same instrument to an agentic knowledge-graph question-answering system built so that each of its components can be removed cleanly, and reads all three sources of the gain.

The instrument is **Agentic Graph Reasoning** (AGR), an explicit state machine over a constrained, deterministic graph tool API. A program orchestrates the loop, and the language model is consulted only at bounded decision points. Its four removable components are a *planner* that decomposes the question into an ordered chain of sub-objectives, a hybrid *edge scorer* that blends embedding similarity with a batched model judgement, a *backtracker*

that restores an earlier frontier and bans the edges that led away from it, and a *structural verification layer* that decomposes each draft answer into atomic claims and checks them against the traversed triples before the answer is emitted. The last is the component the system was designed around, and we state its reach at the width the measurements support. Its two graph routes ask whether an edge joins a claim’s endpoints, not whether it is the edge the claim asserts, so a claim that X is Y’s mother is certified by an edge recording that X is Y’s child. Only the traversed-adjacency route attaches evidence. Sections 3.2 and 8.1 bound it further.

The benchmarks are WebQSP [9] and ComplexWebQuestions [10], 400 questions from each, over a Freebase-derived environment of 2.59 million entities and 8.31 million triples. On them AGR reaches Hits@1 of 0.755 and 0.522 and leads four baselines that share its backbone and its call budget. That number is where the analysis starts rather than where it ends. Against the agentic baseline, a reimplementaion of Think-on-Graph [5] on the same tools, the entire margin comes from the 29% and 44% of questions on which the shared 25-call cap truncates the baseline. Where the cap lets it finish, it is ahead (Section 5.3). The ablation finds one component with a detectable effect, and the effect runs the wrong way. Removing the planner improves WebQSP F1 by 0.083 while cutting tokens by 31%, because decomposing a question with no compositional structure discards the context that would have answered it. Backtracking, model scoring, and claim verification show no effect at the power available, and Section 6.3 states the smallest effect each test could have caught. The failure census finds label defects on 5.5% and 4.8% of the sampled questions, and names a failure that every grounding check passes and that consensus-based label repair cannot see.

Contributions. This paper contributes:

1. A **controlled attribution protocol** for agentic KGQA: one environment, one frozen backbone, one budget meter, metrics and ablation conditions fixed before any test question ran, paired significance tests, and a minimum detectable effect reported beside every null (Sections 4 and 6.3).
2. **Component attribution** for a four-component agentic system. Removing the planner *improves* accuracy on the shallower benchmark while reducing token use, and

the per-stratum breakdown gives the mechanism. The other three components, the claim-verification layer among them, show no detectable effect, and the nulls are reported at the same length as the effect (Section 6).

3. The **location of the aggregate margin** over the agentic baseline. It comes entirely from budget truncation, the candidate-width confound that bounds the other half of the split is measured, and the split is read within hop strata. The reading is affordability under a fixed spend, not resolution quality (Section 5.3).
4. A **hand-read failure census** that names the *echo attractor*, reports which systems converge on it, and quantifies label defects in both benchmarks together with their effect on the accuracy tables (Section 7).
5. **AGR** itself, released with its prompts, per-question records, and analysis scripts, and its verification layer reported as it behaves: how often it fires, how far the record can attribute an acceptance to a route, and what the output contract does and does not guarantee (Sections 3 and 8.1).

Section 2 positions the study against prior work. Section 3 specifies the instrument. Section 4 describes the environment and the protocol. Section 5 reports the aggregate measurements and locates the margin. Section 6 attributes the measurements to components, Section 7 reads what remains, and Section 8 states the limits.

2. Related Work

Three lines of work bear on question answering over knowledge graphs: what retrieval fixed before reasoning can reach, what interleaving retrieval with reasoning buys, and what checking an answer against evidence adds. A fourth subsection names what none of them establishes: which component of an agentic system is responsible for its behaviour. That gap is the one this paper addresses, and the first three subsections lead to it.

2.1. Static Retrieval over Graphs

Retrieval-augmented generation [2] attaches a language model to an external corpus. The system embeds the question, retrieves similar passages, and lets the model answer from them. Where the answer sits in one passage this works well. Where it must be composed

across passages, the ranking signal measures the wrong thing, because the passage carrying the intermediate entity resembles neither the question nor anything the system yet knows to look for. Surveys of the graph–language-model intersection identify this as the central limitation of flat retrieval [11].

Knowledge graphs make the missing structure explicit. They store facts as typed edges between identified entities, so composition becomes a traversal. GraphRAG [4] builds a graph over a corpus and retrieves structured context in place of passages. Related systems extend the idea to long-term memory [12] and to temporally-qualified facts [13]. A second family keeps the retrieval static but learns it: GNN-RAG [14] scores candidate subgraphs with a graph neural network, and SubgraphRAG [15] retrieves a bounded subgraph with a light-weight scorer and shows that a strong model reasoning over it is hard to beat. All of them retrieve once, before reasoning, so the retriever cannot know what the second hop will need. A fixed-radius neighbourhood is either too small to contain the answer or so large that the relevant edges are lost among thousands of irrelevant ones.

2.2. *Agentic Navigation*

Interleaving retrieval with reasoning removes that constraint. The model takes one step, observes what it found, and decides where to look next. This inherits the tool-use and self-reflection patterns established for language agents generally [16, 17, 18].

Think-on-Graph [5] established the pattern for knowledge graphs with model-guided beam search over relations and entities. Its successor, Think-on-Graph 2.0 [19], extends the walk with document retrieval at each step. Plan-on-Graph [6] added two mechanisms to the beam search: decomposition into sub-objectives, and backtracking driven by reflection. AGR’s planner and backtracker are those two mechanisms, re-hosted in a program-orchestrated loop, and its verification layer is the addition; Section 3 names the differences in the mechanics, and Section 8.2 records that Plan-on-Graph itself is not among the systems run here. Reasoning-on-Graphs (RoG) [7] takes a different route. A fine-tuned model generates grounded relation paths as explicit plans, and the system traverses them, so the answers are faithful by construction. Graph-constrained reasoning [20] reaches the same guarantee by constraining decoding to paths in the graph. RoG is also the surveyed system whose data distribution this paper shares: the environment of Section 4.1 is built

Table 1: The design space, on the mechanisms this paper ablates. *Trained* means the system’s own parameters are tuned on the benchmark’s training split. Only the last two rows were run under this paper’s protocol; the rest are read from the cited papers.

System	Decomposition	Edge choice	Backtrack	Output check	Trained
RoG [7]	relation-path plan, generated once	path retrieval	none	faithful by construction	yes
GCR [20]	none	trie-constrained decoding	none	faithful by construction	yes
KBQA-o1 [8]	MCTS over agent actions	reward model	tree search	none	yes
Plan-on-Graph [6]	sub-objectives, in prompt	model pruning	reflection	none	no
Think-on-Graph [5]	none	model pruning (beam)	none	none	no
AGR (this work)	sub-objectives, as data	embedding + model blend	ban-list restore	claims against traversal	no

from its published subgraphs, and Section 5.4 compares the two directly. Other systems add tool abstractions [21], cope with incomplete graphs [22], or generalise across structured sources [23]. Recent surveys treat the convergence as a research programme in its own right [24, 25].

Table 1 places these systems and AGR on the four mechanisms this paper ablates, and on whether the system is trained on the benchmarks. The rows are read from the papers cited; only AGR and the Think-on-Graph reimplementations were run here.

KBQA-o1 [8] is the closest system to AGR’s search strategy, and the most important to distinguish from it. It performs agentic knowledge base question answering with full Monte Carlo Tree Search: a policy model proposes actions, a reward model scores them, and rollouts propagate their values back through the tree. That search wraps a ReAct-style agent generating logical forms against Freebase, and the exploration also produces annotations for incremental fine-tuning. AGR does none of this. Its search is guided best-first with a bounded beam and backtracking, and it carries neither rollouts nor value backpropagation.

2.3. Verification and Self-Correction

A parallel line checks generated content after the fact. Chain-of-Verification [26] has a model draft a response, plan verification questions about it, answer those independently, and regenerate. Multi-agent debate [27] reaches a related effect by having several instances critique one another. Both check against the model’s own knowledge rather than an external symbolic source, and the limits of models correcting themselves without such a source are documented [28]. A second group critiques the *retrieval*. Self-RAG [29] trains the model to emit reflection tokens judging whether a passage supports its output, and corrective RAG [30] grades retrieved documents and triggers a fallback search when they are poor. Hu et al. [31] demonstrate a self-correcting agentic graph pipeline in a clinical setting. FActScore [32] decomposes generations into atomic facts for evaluation, which is the same decomposition applied as a metric rather than as a control step.

The verification layer of Section 3.2 sits between these lines. It decomposes a draft into claims, as the first group does, and checks them against an external source, as the second does, but the source is the set of triples the agent itself traversed, and the check runs before the answer is emitted. The path-faithful systems [7, 20] make the question moot by never generating a claim the graph did not produce. To our knowledge no graph-navigating agent that generates its answer freely checks that answer’s claims against its own traversal before emission. Whether that check earns its place is what Section 6 measures, and the answer is not the one the design anticipated.

2.4. The Attribution Gap

The gap that motivates this paper is methodological. Agentic systems are assemblies of decomposition, scoring, backtracking, and self-correction, and the literature reports them as wholes. When such a system outperforms a static baseline, the published work rarely establishes which mechanism produced the gain, whether some contribute nothing, or how much of the gain is the budget regime the comparison ran under. Later work then inherits the entire assembly on faith, along with its cost.

The exception is instructive. KBQA-o1’s own ablation removes MCTS while leaving every other component intact, and GrailQA F1 drops from 78.5 to 48.5 [8]. That is a thirty-point attribution to a single mechanism, obtained precisely because the authors ablated

rather than reporting the assembly whole. Section 6 applies the same instrument to AGR, with paired tests and a stated minimum detectable effect, and reports the components for which it detects nothing at the same length as the one for which it does. Section 5.3 adds the second source of a reported gain, the budget, which an ablation cannot see.

3. The Instrument: AGR

AGR inverts the usual agent pattern. It does not hand the model a set of tools through native function-calling and let the model drive the loop. A program orchestrates every step, calls the graph tools deterministically, and consults the model only at bounded decision points, through prompts constrained to return JSON. Three consequences follow. Every graph operation is logged by construction rather than reconstructed from the model’s account of what it did. Budgets are enforced by counters a router checks, not requested in a prompt the model may ignore. And a malformed model response degrades one decision rather than corrupting control flow: every call is wrapped in a parser that, on failure, spends one metered retry asking the model to repair its own output before falling back. A fourth consequence is the one this paper uses. Because the loop is a program, each of its four components can be removed by a configuration flag without touching the others, and Section 3.4 says exactly what each flag removes. The rest of this section specifies the loop that produces a traversal (Section 3.1), the layer that checks a draft against it before an answer is emitted (Section 3.2), and the budgets that bound the whole loop’s cost (Section 3.3).

3.1. State Machine and Tool API

Figure 1 gives the topology: six agent nodes over a shared typed state, with three cycles, and Algorithm 1 gives the loop. The **Planner** decomposes the question into an ordered chain of at most four sub-objectives in one model call. Its prompt carries four exemplars, the first of which is a single-hop question that is *not* decomposed, and a later objective may reference an earlier one’s result. **Explorer** and **Evaluator** form a search loop. The explorer expands the frontier, and the evaluator judges whether the active sub-objective is satisfied. The **Backtracker** restores an earlier, higher-scoring frontier and bans the edges that led away from it. That ban list is what turns backtracking into a search operation. Without it, a

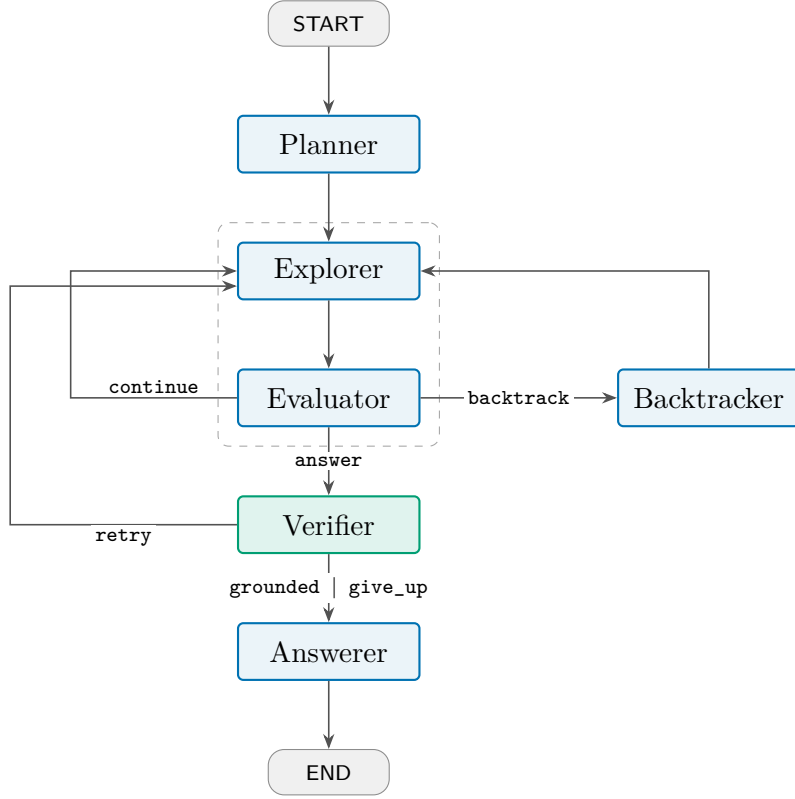


Figure 1: The AGR state machine. The dashed region is the Explorer $\leftrightarrow$ Evaluator search loop. Evaluator $\rightarrow$ Backtracker $\rightarrow$ Explorer restores an earlier frontier, and Verifier $\rightarrow$ Explorer is verification-driven re-exploration. The budgets of Section 3.3 bound all three cycles. Every path from Evaluator to Answerer passes through the Verifier.

deterministic scorer facing the same restored frontier would select the same edges again. Two properties of the topology matter beyond the nodes themselves. It contains cycles rather than running once through, because exploration continues until the evaluator is satisfied and verification can send the agent back to explore. And every path from Evaluator to Answerer passes through the **Verifier**. No route reaches the Answerer without it.

Five graph operations back this loop. They are parameterised Cypher templates against a Neo4j property graph, not free-form queries the model would write itself, because a generated-query failure is indistinguishable in the results from a reasoning failure. `search_entity` resolves a surface form to a node through three tiers, exact name, then full-text index, then vector similarity, and stops at the first that returns a match. `get_relations` returns the relation types incident on an entity with their direction and fanout, after a blocklist

Algorithm 1 The AGR loop. Model calls are the four named procedures and the scoring call inside eq. (1); every other step is deterministic. Constants are the ones the reported runs used.

Require: question q ; annotated topic entities A_0 ; budget configuration (Table 2)

```

1:  $P \leftarrow \text{PLAN}(q)$   $\triangleright \leq 4$  sub-objectives;  $[q]$  alone when the planner is removed
2:  $A \leftarrow A_0$ ;  $T \leftarrow \emptyset$ ;  $S \leftarrow []$ ;  $\text{ban} \leftarrow \emptyset$ ;  $\text{depth} \leftarrow 0$ 
3: loop
4:    $o \leftarrow$  active sub-objective of  $P$ , earlier results substituted for its references
5:    $C \leftarrow \bigcup_{a \in A} \text{get\_relations}(a)$   $\triangleright \leq 300$  relation rows per anchor, by fanout
6:   score each  $c \in C$  by eq. (1); keep the 3 best per anchor not in  $\text{ban}$ 
7:   push  $(A, \text{depth}, \text{max score})$  onto  $S$ ;  $\text{depth} \leftarrow \text{depth} + 1$ 
8:    $T \leftarrow T \cup \text{get\_neighbors}(c)$  for each kept  $c$ ;  $A \leftarrow$  the  $\leq 5$  best new neighbours
9:    $(d, R) \leftarrow \text{EVALUATE}(o, \text{top}_{30}(T, o))$   $\triangleright d \in \{\text{answer}, \text{continue}, \text{backtrack}\}$ 
10:  if depth or time budget is spent then break
11:  else if a backtrack trigger fires then  $\triangleright$  dead end; score  $< \tau$ ;  $d = \text{backtrack}$ 
12:    break unless the backtrack budget allows
13:     $(A, \text{depth}) \leftarrow \text{POPBEST}(S)$ ;  $\text{ban} \leftarrow \text{ban} \cup \{\text{edges of the last expansion}\}$ 
14:  else if  $d = \text{answer}$  and  $o$  is the last objective then break
15:  else if  $d = \text{answer}$  then mark  $o$  done;  $A \leftarrow R$   $\triangleright$  next objective becomes active
16:  end if
17: end loop
18:  $(\text{draft}, E, K) \leftarrow \text{DRAFT}(q, \text{top}_{60}(T, q), \text{candidates})$   $\triangleright$  prose, answer entities, atomic claims
19: route each  $k \in K$ : traversed adjacency, then verify\_connection, then ENTAIL
20: if no claim is unsupported then emit  $\text{draft}$ ,  $E$ , and the triples that supported  $K$ 
21: else if verify, depth, and time budgets remain then append a repair objective for the first unsupported claim;  $A \leftarrow$  its head entity; resume at line 3
22: else filter  $E$  deterministically; one REWRITE call; emit
23: end if
```

removes schema relations, sorted by descending fanout and cut at 300 rows. **get_neighbors** returns at most 200 neighbours along one relation; unnamed mediator nodes, which encode n -ary facts and make up 63.7% of the graph (Section 4.1), are traversed transparently, so a mediator path costs one hop. **verify_connection** asks whether two entities are adjacent directly or through one mediator, ignoring the relation. The fifth, **verify_triple**, was built first to check a claim $\langle h, r, t \rangle$ by asking whether that exact relation held between the two entities. It did not survive contact with generated text. Claim decomposition phrases relations in English (“played for”, “is the capital of”), and those phrases rarely correspond to a single Freebase predicate, so matching on them rejected claims that were true. **verify_connection** replaced it, and nothing in the running loop calls **verify_triple**.

The explorer gathers every relation row of every anchor into one candidate set, at most five anchors, and scores it in one pass. Each row c , an anchor together with a relation and

a direction, receives

$$\text{score}(c) = \alpha \cos(v_{\text{rel}(c)}, v_o) + (1 - \alpha) \text{LLM}(c), \quad (1)$$

which blends two signals at the frozen $\alpha = 0.7$. The first is the cosine similarity between the candidate relation’s embedding and the embedding of the active sub-objective o , both from all-MiniLM-L6-v2 [33], with the relation vocabulary of 7,058 types embedded once in advance. The objective, not the original question, is what gets embedded, and that substitution is the concrete mechanism through which decomposition is meant to help. The second is a batched model judgement, one call, over the twenty candidates with the highest embedding score; every other candidate receives zero for that term. The blend is applied after the model call rather than inside the prompt, so that a prompt free of any configuration value lets an α sweep reuse one set of cached responses across every condition. Setting $\alpha = 1$ disables the model term entirely and makes no scoring call, which is the ablation Section 6 reports. The three best rows per anchor, skipping banned ones, are expanded; the neighbours they return become traversed triples, and the five with the highest reaching score become the next frontier.

The evaluator is one model call. It sees the active sub-objective, the thirty traversed triples most similar to it by the same embedding, and the remaining depth and backtrack budget, and it returns one of three decisions together with the entities it considers to satisfy the objective. Those entities are accepted only if they are names in the traversed set, so the evaluator cannot introduce an entity the graph did not return. When it marks an objective done, its resolved entities become the anchors for the next objective, and references to the earlier objective are rendered with those names. The router after it applies three tests in order. If the depth or the wall-clock budget is spent, the loop ends and the verifier runs. Otherwise, if a backtrack trigger fires and the backtrack budget allows, the backtracker runs. Three triggers exist: the last expansion added no triple, the best score of the last expansion fell below $\tau = 0.20$, or the evaluator asked to backtrack. Otherwise the evaluator’s decision stands. The backtracker pops the snapshot with the highest recorded score from the stack, which is often not the most recent one, restores its anchors and depth, and bans the anchor–relation–direction rows the most recent expansion followed. The mismatch between the snapshot it restores and the rows it bans is deliberate in neither direction, and Section 6.2

measures what it costs. This is where the mechanics part from Plan-on-Graph [6], whose decomposition and reflection live in prompts the model executes. Here the plan is a data structure the program advances, two of the three backtrack triggers are deterministic tests on the scores, and the ban list, not a reflection, is what makes a restored frontier explore differently.

3.2. Structural Verification Layer

The navigation loop above ends with a set of traversed triples and a set of accumulated candidate entities. Nothing in it constrains what the final answer *asserts*. The verifier is the check that does. A single model call sees the question, the sixty traversed triples most similar to it, and up to twenty-five candidates. It drafts prose, extracts the entities the draft asserts as its answer, and decomposes the draft into atomic claims $\langle h, r, t \rangle$ over entity *names*, since the model has never seen a graph identifier. The relation slot is left as free text. Asking the model to name a Freebase predicate would fail as unreliably as `verify_triple` did.

Each claim is routed through up to three tests, and it leaves at the first one that settles it (Figure 2). **Traversed adjacency** collapses the traversed triples into an undirected set of endpoint pairs. A claim is supported if its pair is in that set, and every traversed triple joining the pair is attached as citable evidence. This is the only route of the three that attaches evidence. **verify_connection** catches claims whose endpoints were both reached but never walked together. It queries the full graph for the same relation-agnostic adjacency. Claims that clear neither route, together with those naming an entity the agent never traversed at all, fall to an **entailment check**: one batched model call judging each remaining claim against the same fact window the drafter saw. Symbols decide, and the model is consulted only where the graph is silent. Both graph routes ignore the relation and the direction, so what they establish is that the endpoints of a claim are connected in the environment, not that the claimed relation is the one the edge records. Section 8.1 states the exposure that follows and how much of it the record can bound.

The router that follows has three outcomes. No unsupported claim routes to **grounded**, and the draft is emitted verbatim with no further model call. An unsupported claim with verification, depth, and time budget remaining routes to **retry**: the verifier appends a repair

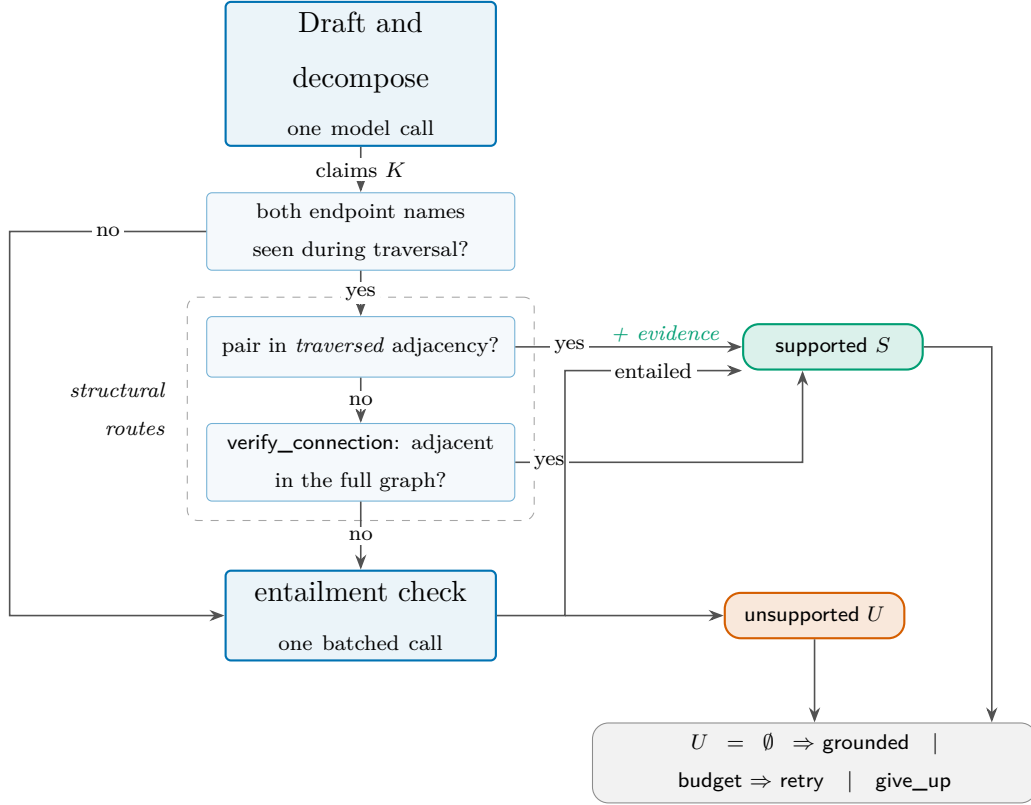


Figure 2: The path one atomic claim takes through the verification layer. Three tests are tried in a fixed, cheap-to-expensive order, and a claim leaves at the first that settles it. Only traversed adjacency attaches citable evidence. A claim naming an entity the agent never reached skips both structural routes entirely.

objective naming the first unsupported claim, anchors the frontier on that claim’s head entity, and control returns to the Explorer. Otherwise the router takes `give_up`. One rewrite call then restates the draft around a *deterministically filtered* entity list. An entity survives if it appears in a supported claim or in no unsupported claim. An earlier design let the model regenerate the list itself, and that reintroduced exactly the ungroundedness the layer exists to prevent.

One development-set question shows every route firing. Asked which university, founded in 1701, actor James Franco attended, the agent walks the education relations from the actor and never satisfies the founding objective, because the environment holds no date literals. The draft names **University Yale**, the graph’s spelling, and decomposes into an attendance claim and a founding claim. The attendance claim’s endpoints are in the traversed adjacency set and four traversed triples are attached as evidence. The founding claim names **1701**, which no triple mentions, so it falls to the entailment check, which correctly reports it unsupported. The depth budget is spent, so the router takes `give_up`, the filter keeps **University Yale** because a supported claim names it, and the rewrite hedges the founding date. The scored answer is a hit. Before deterministic filtering replaced a second generation call, the identical trace ended with **Yale University**, a string that appears nowhere in the environment.

The layer’s reach should be stated plainly here, because it bears on how the null result of Section 6 should be read. On the 80-question development set used to validate this design, 77 questions asserted no unsupported claim and took the grounded path at zero additional model cost. The remaining 3 took `give_up`, and the retry route did not fire once, because every question that reached the verifier with an unsupported claim had already exhausted its depth budget. On most questions, then, the layer is a pass-through that certifies a draft rather than a mechanism that visibly intervenes. Sections 5, 6 and 8.1 report what it does at test scale.

3.3. Budgets and Termination

Table 2 lists the caps that bound every run. Routers and the model client enforce them; no prompt requests them. Three of them (depth, wall-clock, and verification iterations) are checked at two separate sites each, because the `retry` edge re-enters the Explorer without

Table 2: Budget configuration. Every row above the rule is hashed into the run configuration, so a reported result can always be traced to the caps that produced it. The two rows below the rule are tool-layer caps that bound retrieval rather than spend.

Parameter	Value	Enforced in
max_depth	4	post-evaluation <i>and</i> post-verification routers
beam_width	3	Explorer (per-anchor slice)
max_anchors	5	Explorer <i>and</i> Evaluator (frontier slice)
max_backtracks	3	post-evaluation router
max_llm_calls	25	model client, before every call
max_verify_iters	2	post-verification router <i>and</i> the verifier
max_seconds	300	post-evaluation <i>and</i> post-verification routers
max_relations	300	get_relations: candidate relations offered to the scorer, sorted by fanout
max_fanout	200	get_neighbors: neighbours returned per expansion

passing back through the router that would otherwise catch it. The two rows below the rule are not budgets but tool-layer caps. They bound what the agent can see rather than what it can spend, and Section 4.2 shows why they matter for the comparison.

Every cycle in Figure 1 therefore passes through a router that checks or decrements a monotone counter. So no sequence of model outputs, however adversarial, can produce a non-terminating run. The cost of enforcing this is small in the common case. The grounded path above adds nothing beyond the single drafting call already required to produce a draft. At test scale, removing the layer altogether saves 2% and 6% of tokens on the two ablation half-splits (Table 8). Whatever the case for or against the layer, it is not a cost argument.

3.4. Removing a Component

Each ablation condition of Section 6 flips one flag and leaves every other line of the loop as it is. **No planner** replaces the planning call with a plan of one objective, the

question itself; anchors still come from the annotated topic entities, as they do in the full system. **Embedding-only scoring** sets $\alpha = 1$, which makes no scoring call and ranks candidates by cosine similarity alone. **No backtracking** sets the backtrack budget to zero, so the router ends the search where it would have backtracked. **No verification** leaves the drafting call unchanged, entity list and claim decomposition included, but checks no route, makes no entailment call, and emits the draft verbatim. That condition therefore measures the checking and not the drafting, and its token saving is the cost of the routes rather than of the draft. The budget configuration is hashed into every record, so the condition that produced a row of Table 8 can be recovered from the row.

4. Experimental Setup

Everything in this section exists to make one inference safe: that a difference between two systems is a difference between architectures. We describe the environment all five systems share first, then the benchmarks, the four baselines, and the single frozen backbone every one of them calls, and finally the metrics and the protocol fixed before any test question was run. Where something is *not* held constant, we name it here rather than leave it for a reader to find.

4.1. Knowledge Environment

Every dataset this paper consumes is public, standard, and third-party. The Freebase [3] snapshot and the two question sets all arrive through one distribution: the per-question subgraphs published with Reasoning-on-Graphs [7], released as `rmanluo/RoG-webqsp` and `rmanluo/RoG-cwq`. The datasets are used unmodified. What is built here is the environment derived from them: a property graph of 2.59 million entities and 8.31 million triples over 7,058 distinct relation types, assembled as the union of every per-question subgraph across both datasets and all three splits. All five systems navigate that one graph. Just under two-thirds of its nodes (63.7%) are unnamed mediators encoding n -ary facts. That is why the tool layer traverses them transparently rather than exposing them as candidate entities (Section 3).

An environment assembled this way is friendlier than full Freebase, and we state the consequence rather than leave it to be inferred. Before any system ran, a validation

gate measured what fraction of questions have at least one gold answer reachable from a topic entity within the four-hop cap: 97.0% on the WebQSP sample and 99.2% on the ComplexWebQuestions sample. A sample that passed that gate is what the paper calls *certified*. That ceiling is why a failure in Section 7 can be attributed to the system rather than to a missing edge. Almost every question is answerable in principle, so a wrong answer is a navigation or selection error, not an absent fact. It is equally why **absolute accuracies here do not transfer to a full-Freebase setting**. Every comparison in Section 5 is between systems traversing the identical environment, which is what the comparison is for. None of the numbers should be read against published figures obtained over a different substrate; Section 5.4 shows what such a reading looks like when it is done with the caveats attached.

One limit of the environment concerns expressiveness rather than coverage. Every node carries a single entity label. There is no typed literal, so a date or a number exists only incidentally, as an entity whose name happens to be a numeric string, and the tool API exposes no ordering or aggregation operator. A question whose gold answer is a date, a measured quantity, or a superlative over candidates is therefore unanswerable in this environment by construction. Section 7 reports how much of the residue that class accounts for, and Section 8 carries it as a scope limit.

4.2. Benchmarks, Baselines, and Backbone

We evaluate on WebQSP [9] and ComplexWebQuestions [10], the two standard multi-hop KGQA benchmarks over Freebase. **We sample 400 questions from each** rather than running the full 1,628 and 3,531 test splits. That was a pre-specified fallback, adopted when the full-split cost was projected at roughly three times the budget available for the work. The sample is stratified by hop count at a fixed seed, so its strata are proportional to the split’s: 256, 128, and 4 questions at one, two, and three or more hops on WebQSP, with 12 whose gold answer the gate found unreachable, and 137, 211, and 49 on ComplexWebQuestions, with 3 unreachable. The resulting 95% bootstrap intervals are roughly ± 4 to ± 5 points on Hits@1. Section 8 says which of the paper’s claims survive that width and which are reported as directions rather than effects. The two benchmarks differ less in the typical question than in the tail. Exactly half of the WebQSP sample carries a single gold answer, but its mean

is 7.15 gold entities per question and its largest question lists 237. ComplexWebQuestions averages 1.75 with a median of 1. That tail is one reason we report F1 beside Hits@1.

Four baselines span the design space. Each is a paradigm rather than a tuned competitor, and three of the four are controls: they bound what a paradigm can do on this environment, and nothing in this paper rests on beating them. A **no-retrieval** control answers from parametric memory alone, establishing what the backbone knows without the graph. **Vector-RAG** retrieves verbalised triples by embedding similarity. **Static GraphRAG** [4] fetches a fixed-radius neighbourhood around the linked topic entities in one shot, reranks it by embedding similarity to the question, and answers from the top 30 facts. Two properties of that implementation bound what it licenses, and we state them here rather than leave a reader to infer them from its scores. The radius is *one logical hop*. Mediators are traversed transparently, so a mediator path costs two graph edges while remaining one hop, and a genuine two-hop chain is outside its reach by construction. And expansion retrieves at most 100 edges per topic entity with no ordering criterion, so on a high-degree entity the retained edges are an arbitrary sample rather than the best or the first. At least one topic entity is truncated on 72.5% of the 800 test questions. This baseline therefore licenses one narrow claim: *retrieval fixed before reasoning must commit to a radius in advance, and any fixed radius is wrong for some question in the set*. It does not license a claim about what a two-hop static retriever would score. That experiment was not run.

Think-on-Graph [5] is the agentic comparator and the one comparison in this paper between two systems of the same paradigm. It performs model-guided beam search over relations and entities, and it is a reimplementing on this environment’s tools rather than a quoted number. Beam width and depth follow the original paper, 3 and 3. At each depth it makes one relation-pruning call per frontier entity, one entity-pruning call per selected relation, and one sufficiency call, so a full depth at width 3 costs up to thirteen calls, and the 25-call cap of Table 2 admits one full depth and part of a second. Two widths beneath the beam are this reimplementing’s own rather than the original’s: it offers the pruning prompt the first 40 relations of an entity and the first 20 neighbours of a relation, against the 300 and 200 AGR sees (Table 2). Reimplementing is a stronger comparison than citing the original figures, because graph, model, budget, and scoring rule are held constant and

only the algorithm varies. It also means its numbers are not comparable to the published ones, and we do not present them as such. All baselines were debugged on development data and certified before any test question ran, against a pre-specified diagnostic targeting the one failure that would be an implementation defect rather than a paradigm limit: retrieval found the gold answer and the system hedged anyway.

Every system shares one frozen backbone, `gpt-5.4-mini-2026-03-17`, a dated snapshot rather than an alias, at temperature 0.0 with reasoning effort disabled. A check that logs the reasoning-token count on every call read zero across every run in this paper. The systems also share the entity resolver, the question files, the output schema, and *one budget meter with the same 25-call cap*. Both graph-navigating systems seed from the datasets’ annotated topic entities, which removes entity linking as a confound. Holding all of this constant is what licenses the paper’s central inference: a difference between two systems is attributable to architecture, not to model capacity, retrieval corpus, or spend. Three things are deliberately not held constant, and we name them here rather than bury them. The baselines receive a plain answering prompt, while AGR’s drafting prompt carries the grounding provisions that are part of the system under test. The two navigators consider candidate sets of different widths, the 40 and 20 above against 300 and 200. And the shared call cap binds on the two navigators at very different rates, which Section 5.3 shows is where the aggregate margin lives. The second is where a reader should look first for a confound, and Section 5.3 reports the measurement that bounds it.

4.3. Metrics and Protocol

Accuracy is reported as **Hits@1** and per-question **F1**. Both systems and baselines return an unranked set of answer entities, and Hits@1 here follows the benchmarks’ convention for that case: a question scores a hit if any returned entity is in the gold set. We report F1 because a system can score a hit on that rule while asserting several wrong entities alongside the right one, and Section 5.3 checks directly whether AGR buys hits by naming more entities. A **hedge** is an empty predicted entity set. Hedges score zero on both metrics, and the hedge rate is reported as its own column, so a system buying precision through abstention is visible rather than flattered. Hits, hedges, and wrong answers always partition the whole set. Nothing is dropped from the denominator, including questions the environment provably

cannot answer. Matching is exact after Unicode normalisation, case folding, and whitespace stripping, with no fuzzy or substring matching. We measured the obvious worry rather than assumed it away: of AGR’s wrong answers, 1 of 65 on WebQSP and 6 of 99 on ComplexWebQuestions are surface-form near-misses that aggressive normalisation would have scored as hits. The failure census of Section 7 is reading real errors.

Groundedness is measured at two levels. The structural tier is deterministic and applies to every asserted entity in all 4,000 records: an entity is grounded if and only if it exists as a node in the environment and connects to a topic entity within four hops. The semantic tier asks the harder question of whether the entity is supported *as the answer*. For a stratified sample of 60 answers per system per dataset, it retrieves the graph’s own shortest connecting paths and has a language model judge whether any path expresses the relationship the question asks about, under the governing instruction that mere connectedness is not support. That judge was validated against a human annotator on 100 items, reaching 85% agreement and Cohen’s $\kappa = 0.6995$. That *misses* the 0.70 we pre-specified, by 0.0005, and Section 8 treats the miss as a limitation rather than rounding it away. Because path retrieval takes only the three shortest paths, semantic-tier rates are conservative lower bounds. They are depressed uniformly across systems and read as a between-system comparison rather than as absolute support rates. Efficiency is tokens and model calls, both exact from the shared meter. Wall-clock time per question was recorded as well, and Section 5.2 reports it with the caveat it needs.

The protocol was **pre-specified**. Research questions, metrics, baseline certification thresholds, and ablation conditions were fixed and documented before the test sets were constructed, and no hyperparameter was tuned on test. Four decisions in particular were written down before the measurements they govern: the frozen configuration $\alpha = 0.7$ and $\tau = 0.20$ of Section 3, selected on a disjoint 80-question development set and never revisited; the sample size of 400 questions per dataset, named as the fallback before the cost was known; the baseline certification threshold; and the judge threshold of $\kappa \geq 0.7$. The last was missed, and it is reported as missed. A threshold that counts only when it is met is not a threshold. Nothing was filed with a public registry, so the claim is pre-specification and not pre-registration. Confidence intervals are 95% percentile bootstrap intervals over questions

from 10,000 resamples, and paired comparisons use the exact form of McNemar’s test on per-question correctness. The thirteen prompt templates the five systems send, AGR’s seven and the baselines’ six, are reproduced verbatim in Appendix A, generated from the code by a script so that they cannot drift. Deviations encountered during execution, and there were several, are recorded in the repository rather than narrated here.

5. Aggregate Results and Where the Margin Lives

The same runs are read several times over. This section reports what the five systems answer correctly and what they spend doing it, then asks where the margin over the agentic baseline comes from, how the numbers sit against a published system on the same question sets, how accuracy varies with the number of hops a question requires, and whether the entities the systems assert are supported by the graph they navigated. The aggregate table comes first because the rest of the paper takes it apart. Where a pass narrows to two systems it says so. Where AGR is not the best of the five, it says that too.

5.1. Accuracy

Table 3 gives the main matrix. AGR has the highest Hits@1 and F1 in every column. It reaches Hits@1 of 0.755 on WebQSP against the agentic baseline’s 0.660, and 0.522 on ComplexWebQuestions against 0.435. The margin over Think-on-Graph is 9.5 and 8.7 points. The per-question F1 gap is wider still, 0.642 against 0.477 and 0.469 against 0.378, because F1 penalises the extra wrong entities that a set-intersection hit forgives. All eight paired comparisons against the four baselines reject at the conventional level. The two against Think-on-Graph, the only comparisons in which both systems navigate the same graph with the same model under the same call budget, rest on 76 questions AGR alone answers against 38 the other way on WebQSP ($p = 4.8 \times 10^{-4}$), and 86 against 51 on ComplexWebQuestions ($p = 3.5 \times 10^{-3}$). That is the aggregate. Section 5.3 shows that it is a statement about a budget regime, and Section 6 shows which of its components earn their place.

Two features of this table cut against a simple reading. **Raw hits mislead on the static graph baseline, and the correction is itself a finding.** GraphRAG scores 0.340 on WebQSP, below the no-retrieval control’s 0.453. That invites the conclusion that

Table 3: Main results, 400 questions per dataset, one frozen backbone and one 25-call budget for every system. Hedge is the proportion of questions answered with an empty entity set. The three single-call systems are controls that bound a paradigm; the comparison the paper reads is the one between the two navigators.

System	WebQSP				ComplexWebQuestions			
	Hits@1	F1	Hedge	Calls	Hits@1	F1	Hedge	Calls
no-retrieval	0.453	0.305	12.2%	1.0	0.307	0.279	38.0%	1.0
Vector-RAG	0.568	0.432	27.0%	1.0	0.203	0.168	48.2%	1.0
GraphRAG	0.340	0.262	55.8%	1.0	0.205	0.183	60.8%	1.0
Think-on-Graph	0.660	0.477	18.8%	12.8	0.435	0.378	22.5%	18.2
AGR	0.755	0.642	8.2%	6.2	0.522	0.469	23.0%	8.9

the implementation is broken. Putting the wrong-answer counts beside the hits inverts the picture. The control asserts on 351 of 400 questions and is wrong on 170 of them, an *assertion precision* of 51.6%, where assertion precision is the share of questions a system commits on that it answers correctly. GraphRAG asserts on 177 and is wrong on 41, or 76.8%. The mechanism is the prompt. The retrieval baselines are instructed to answer only from the facts retrieved and to return nothing otherwise, so they abstain when retrieval misses. The control has no such constraint and guesses from memory, and on these benchmarks its guesses are often right. Whether that is pretraining exposure or general knowledge we did not measure and do not claim. What the control exposes is that a grounded system can score below an unconstrained one on raw hits while committing four times fewer falsehoods, and that the raw hit rate alone would misreport the comparison.

The second is that AGR does not lead on every column. On ComplexWebQuestions its hedge rate of 23.0% is marginally *above* Think-on-Graph’s 22.5%, so it declines to answer slightly more often there. The accuracy lead is not bought by answering more freely, and assertion precision on that dataset makes the point directly. The two systems commit at effectively the same rate, on 308 and 310 of 400 questions. From those near-identical commitments, AGR is right 209 times and wrong 99, against 174 and 136 for Think-on-Graph: 67.9% assertion precision against 56.1%. On the deeper benchmark, the system with an explicit plan, a guided scorer, and a claim check finds 35 more answers and asserts 37 fewer falsehoods than beam search does, on the same graph, with the same model, from

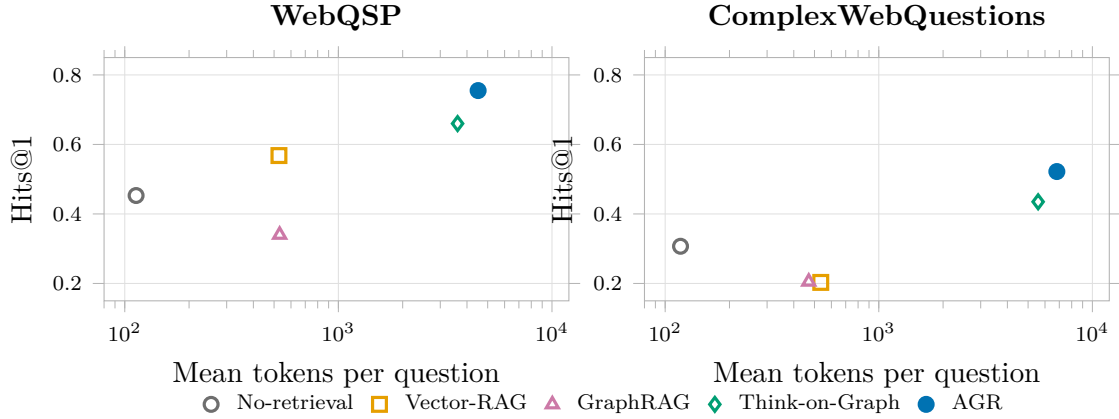


Figure 3: Accuracy against mean tokens per question, both datasets, with cost on a log scale. The single-call baselines cluster at the left. The two navigators sit at the right, with AGR above and to the right of Think-on-Graph on both panels: it buys its accuracy with more tokens per question, not fewer. The halving is in calls, which are not plotted.

the same number of commitments.

5.2. Cost

AGR reaches those numbers at roughly half the model calls of the agentic baseline: 6.2 against 12.8 on WebQSP and 8.9 against 18.2 on ComplexWebQuestions, ratios of 0.48 and 0.49. Figure 3 plots accuracy against cost for all five systems.

The call saving is genuine and the token saving is not. AGR spends 4,511 mean tokens per question on WebQSP against Think-on-Graph’s 3,615, and 6,818 against 5,572 on ComplexWebQuestions. It is the *number of round trips* that halves, not the volume of text. Fewer, larger calls is the accurate description. Which cost binds is a deployment question. On the token axis of Figure 3 Think-on-Graph is a frontier point rather than a dominated one, since it buys lower accuracy at a lower token price. On calls it is dominated on both datasets, and calls are what the shared budget meters. Against one-shot retrieval, agency costs roughly an order of magnitude more in tokens. That is the price of the accuracy in Table 3, and it is stated as a price. Wall-clock time tells the same story in seconds: a mean of 11.2 and 17.2 per question for AGR against 12.9 and 15.2 for the baseline, and between 1.0 and 2.2 for the three single-call systems. Read those as magnitudes only. Records replayed entirely from a response cache are excluded from those means, but between 3.6% and 15.2%

Table 4: AGR against Think-on-Graph, split on whether the shared call cap truncated the baseline. Read from the per-record `budget_exhausted` trace flag.

Subset	<i>n</i>	Think-on-Graph Hits@1	AGR Hits@1
<i>WebQSP</i>			
Think-on-Graph finished	283	0.852	0.788
Think-on-Graph clipped	117	0.197	0.675
<i>ComplexWebQuestions</i>			
Think-on-Graph finished	224	0.629	0.607
Think-on-Graph clipped	176	0.188	0.415

of the two navigators’ calls in the remaining records were still served from the cache, which shortens a cached call to nothing. The figures are therefore lower bounds and do not rank the two navigators against each other at the second.

AGR never reaches the shared call cap. Across all 800 test questions it hit the 25-call limit zero times. That is not the same as running unbounded: its depth cap of Table 2 refuses a deeper expansion on 25.6% of test questions. The baseline reaches the cap far more often, and the next subsection shows that this asymmetry, not resolution quality, is where the aggregate margin lives.

5.3. Where the Margin Comes From

The more informative result is *where* the accuracy margin comes from, and it is not where a reader would assume. Beam search costs width times depth, so the shared 25-call cap binds on Think-on-Graph far more often than on AGR. At width 3 a full depth costs the baseline up to thirteen calls (Section 4.2), so the cap admits one full depth and part of a second, and the reimplementaion reached its third expansion on 6 of 400 WebQSP questions and 11 of 400 on ComplexWebQuestions. Its published depth of three is nominal under this cap. Splitting the questions by whether the cap cut Think-on-Graph off before it finished separates two things the aggregate confounds: how good its search is, and whether it can afford to finish that search. Table 4 gives the split, and it reverses the comparison on one side.

On the questions where the budget lets Think-on-Graph run to completion, it is *ahead* of AGR: 0.852 against 0.788 on WebQSP, and 0.629 against 0.607 on ComplexWebQuestions.

Table 5: The budget split of Table 4 read within hop strata: n and Hits@1 for the two navigators on the questions the baseline finished and the questions the cap clipped. The four three-hop WebQSP questions and the unreachable strata (12 and 3 questions) are omitted.

Dataset	Hops	Think-on-Graph finished			Think-on-Graph clipped		
		n	Think-on-Graph	AGR	n	Think-on-Graph	AGR
WebQSP	1	175	0.846	0.766	81	0.160	0.728
WebQSP	2	100	0.910	0.890	28	0.321	0.679
CWQ	1	69	0.754	0.609	68	0.309	0.309
CWQ	2	123	0.561	0.610	88	0.102	0.477
CWQ	3+	31	0.613	0.581	18	0.167	0.556

The entire aggregate margin comes from the 29% and 44% of questions on which the cap truncates it. There its accuracy collapses to 0.197 and 0.188, while AGR still reaches 0.675 and 0.415.

The split conditions on the baseline’s own outcome, so its two halves need not be equally hard, and the reading above would be unsafe if the clipped half were simply the hard half. Table 5 therefore reads the split within hop strata. On WebQSP the clipped half is not harder for AGR. 81 of the 117 clipped questions are single-hop, AGR scores 0.728 on them against 0.766 on the single-hop questions the baseline finished, and the baseline’s fall from 0.846 to 0.160 across the same two sets is the cap, not the questions. On ComplexWebQuestions the halves do differ: AGR scores 0.309 on the clipped single-hop questions against 0.609 on the finished ones, so there truncation and difficulty are confounded and the affordability reading is correspondingly weaker. The finished halves are not uniform either. On ComplexWebQuestions the baseline leads AGR only on single-hop questions, 0.754 against 0.609, and trails on two-hop, 0.561 against 0.610. That is the planner asymmetry of Section 6.1 seen from the other side.

The correct reading of this is affordability, not resolution quality. Exhaustive beam search finds answers well when it is allowed to finish. Guided search finds them almost as well and finishes within a budget the beam search cannot. On a fixed spend the second is the better architecture. The first would win if spend were unconstrained. That is a claim about operating regimes rather than about reasoning power, and it is narrower than the

aggregate table alone would suggest. The cap of 25 calls is itself a choice, and a larger one would move the split. The clip rates are reported so a reader can see how far, and a sweep of the cap is the first experiment Section 8.4 names.

One asymmetry bounds how far that reading extends, and it is the candidate-width difference named in Section 4.2. Both navigators call the same tools, but they truncate what those tools return at different widths. Think-on-Graph keeps the first 40 relations per entity and the first 20 neighbours per relation, the pruning widths of the reimplemented algorithm, against AGR’s 300 and 200. The relation list is returned sorted by *descending* fanout, so a binding cut discards the low-fanout tail. That tail is where the specific relation answering a specific question tends to sit, since a high-fanout relation is rarely the edge that resolves a particular entity. Over the committed tool logs the 40-relation cut binds on 31.6% of the 1,651 entities Think-on-Graph expanded and the 20-neighbour cut on 32.8% of its 7,992 neighbour calls, against 1 of AGR’s 3,097 expanded entities and 3.3% of its neighbour calls.

The affordability conclusion survives this, because a thinner candidate pool cannot explain why a system runs *out of calls*. What it bounds is the other half of the split. The 0.852 and 0.629 the baseline reaches on the questions it finishes were measured while it searched a narrower pool than AGR. They are therefore a **lower bound** on what an equal-width reimplementation would reach, not an estimate of it. And since the baseline is already ahead of AGR on those questions, equalising the widths would be expected to widen that lead rather than close it. Beam width and depth follow the original paper. The candidate widths beneath them do not, and equalising them is the first change we would make in a follow-up. That re-run would not vary one thing, though. Wider candidate sets make each pruning call dearer, so equal widths would also raise the baseline’s clip rate. That moves the boundary between the two halves of Table 4 at the same time as it changes what the search resolves within each. The two effects have to be read apart, which means reporting that split again rather than an aggregate in which they cancel or compound invisibly.

The breadth of AGR’s answers is checked rather than assumed, because a metric that scores a hit on any match rewards naming more entities. On the WebQSP questions it answers, AGR names 3.37 entities against Think-on-Graph’s 2.56, and its entity precision

Table 6: AGR against RoG’s published figures, in points. RoG’s row is as published in [7], measured over the full test splits. AGR’s row is Table 3, measured over the certified 400-question samples, which are stratified draws from those same splits. The two rows are computed over nested populations under different protocols; the text says what that permits.

System	WebQSP		ComplexWebQuestions	
	Hits@1	F1	Hits@1	F1
RoG (published)	85.7	70.8	62.6	56.2
AGR (this work)	75.5	64.2	52.2	46.9

over those same answered questions is 0.760 against 0.703. On ComplexWebQuestions the figures are 1.54 against 1.40 entities and 0.629 against 0.520 precision. If AGR bought breadth without accuracy, precision would fall as the entity count rose. It does not fall.

5.4. Positioning Against Published RoG Results

Two facts in this paper invite a comparison that the controlled protocol above cannot make. The environment of Section 4.1 is built from the per-question subgraphs that the RoG distribution publishes, and Section 2.2 surveys RoG [7] as the closest published system in the path-retrieval family. A reader who notices both will ask how the two systems compare. The aggregate answer does not favour this work, and Table 6 gives it before the qualifications rather than after them.

RoG leads on all four figures, and the margin is not sampling noise. AGR’s 95% intervals on Hits@1 are [71.3, 79.7] on WebQSP and [47.5, 57.0] on ComplexWebQuestions. RoG’s 85.7 and 62.6 fall outside both. Four differences bound what the table establishes. The first is decisive and the other three are ordinary. **RoG is trained on these benchmarks; AGR is trained on nothing.** RoG’s backbone is instruction-tuned on the training splits of both benchmarks, which hold 2,826 and 27,639 questions in the distribution used here. AGR’s backbone is frozen and prompted, and it has seen no question from either benchmark before evaluation. The comparison is between a supervised system and a zero-shot one, which is a difference in kind rather than in degree. **The backbones differ.** RoG fine-tunes LLaMA2-Chat-7B. Section 4.2 holds the backbone constant precisely because it accounts for much of the spread in cross-paper tables, and it cannot be held constant against a system whose contribution *is* its fine-tuned weights. **The populations differ.** RoG reports over the

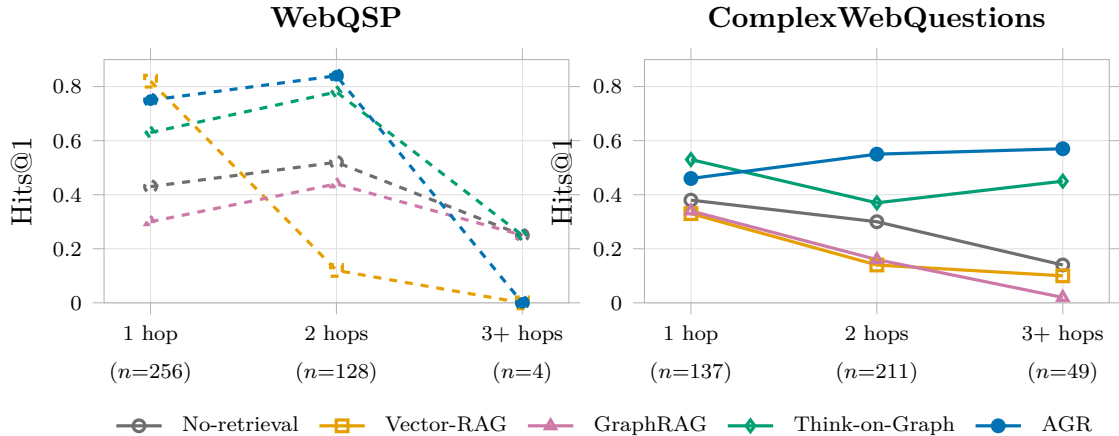


Figure 4: Hits@1 by hop stratum. On ComplexWebQuestions (right) AGR is the only system whose accuracy rises monotonically with hop count. The WebQSP three-hop stratum holds four questions and no weight should be placed on it.

full test splits. Our figures are over stratified 400-question samples of the same splits, which estimate the same quantity, with the intervals above as their uncertainty. RoG publishes no interval, so the gap can be bounded from one side only. **The substrate differs in one respect.** RoG retrieves within each question’s own published subgraph, while AGR navigates the union of all of them, so the two systems see the same triples but AGR sees every other question’s triples as well.

What the comparison settles is AGR’s place in the published landscape: a training-free system reaching 75.5 and 52.2 against a supervised system’s 85.7 and 62.6 on the same question sets, with no in-domain examples. What it does not settle is anything this paper measures. Neither the component-level ablation of Section 6 nor the failure census of Section 7 has a counterpart in RoG’s evaluation, and closing the gap in Table 6 would inform neither.

5.5. Accuracy by Hop Depth

Figure 4 stratifies by the number of hops separating a topic entity from the gold answer.

On ComplexWebQuestions, AGR is the only system whose accuracy *rises* with hop count: 0.46 at one hop, 0.55 at two, 0.57 at three or more, over strata of 137, 211, and 49 questions. The last step sits well inside the interval of a 49-question stratum, so the claim is the direction across the three strata, not the size of any one step. Every other system

ends below where it started. That phrasing is deliberate, and the weaker alternative is the true one. Think-on-Graph does not decay monotonically. It falls from 0.53 to 0.37 and then partly recovers to 0.45, which is a different shape from a steady decline, and we report it as such. Think-on-Graph also beats AGR on that dataset’s one-hop stratum, 0.53 against 0.46. Section 6.1 identifies the mechanism: decomposition costs accuracy on questions that do not need decomposing, and that stratum is full of them.

The static graph baseline’s decay across these strata is deliberately *not* offered as evidence for the paradigm, because its retrieval radius confounds it. A one-hop retriever cannot contain a two-hop chain, so part of that decline measures the radius rather than static retrieval as such. The evidence that it is a radius effect is in the strata themselves. GraphRAG scores 0.44 on WebQSP’s two-hop stratum, where those questions are largely mediator paths a one-hop expansion does reach, against 0.16 on the ComplexWebQuestions two-hop stratum, where the chains are genuine compositions it cannot reach at all. A retriever failing uniformly on depth would not show that gap.

A rising curve is what decomposition would predict: it converts one hard question into several easy ones, so the questions that punish single-shot retrieval hardest are the ones an agent that re-plans between hops should handle best. The curve is consistent with that mechanism rather than evidence for it. Section 6.1 finds that removing the planner costs accuracy on this dataset in the direction the mechanism predicts, and that the loss does not reach significance at the sample available. The WebQSP panel carries no comparable evidence. Its three-hop stratum contains four questions, which is why the claim is scoped to one dataset.

5.6. Groundedness

Across both datasets AGR asserted 1,709 entities, and **zero of them were ungrounded**. Every entity it named exists in the environment and is reachable from a topic entity within the hop cap. The parametric control asserted 1,001 entities, of which 22.1% failed that test. Two qualifications come before any reading of that contrast. The property is close to guaranteed by construction for any system that copies its entities out of tool results, because every node a tool returns is in the environment and was reached from a topic entity. And **it belongs to graph navigation, not to the verification layer**.

Table 7: Semantic-tier support rate: the share of sampled assertions a judge found supported *as the answer* by the graph’s own connecting paths. $n = 60$ per cell, so the 95% binomial interval has a half-width of roughly 12 to 13 points. These are conservative lower bounds by construction (Section 4.3).

System	WebQSP	CWQ
no-retrieval	63.3%	36.7%
Vector-RAG	61.7%	50.0%
GraphRAG	55.0%	36.7%
Think-on-Graph	61.7%	43.3%
AGR	66.7%	48.3%

Think-on-Graph also reaches exactly zero ungrounded assertions, on 1,265 entities, without any claim-checking component whatsoever. Whatever the verification layer contributes, it is not this number. What the comparison establishes is that an agent which can only name entities it retrieved cannot fabricate one, and that this property follows from the retrieval discipline rather than from any check applied afterwards.

The semantic tier asks the harder question of whether an asserted entity is supported *as the answer*, and it separates the systems far less. Table 7 gives every cell rather than a selection, because the ordering differs by dataset and any subset of it misleads.

Every system lands in a 36.7–66.7% band. AGR is highest on WebQSP at 66.7%. **On ComplexWebQuestions it is second**, 5.0 points ahead of the agentic baseline but 1.7 behind Vector-RAG’s 50.0%. Its margin over Think-on-Graph is 5.0 points on both datasets. Its lead over *every* system is a WebQSP result only, and we say so rather than quote the two comparators it happens to beat.

Neither gap survives the sample. At $n = 60$ per cell a 5-point difference sits well inside the interval, as does Vector-RAG’s 1.7, and no conclusion in this paper rests on either. What the tier establishes is the level rather than the ranking, and the level is the more useful finding: **zero structural hallucination coexists with between a third and nearly two-thirds of assertions failing a strict semantic-support test**, for every system, including the two that are structurally perfect. Semantic support is hard for the paradigm, not for one system within it.

Table 8: Component ablations. $\Delta F1$ is ablated minus reference: positive means the system got better without the component. The interval is a paired percentile bootstrap over questions (10,000 resamples); p from McNemar’s exact test on paired per-question correctness.

Removed	WebQSP ($n = 200$)				CWQ ($n = 198$)			
	$\Delta F1$	95% CI	p	Tokens	$\Delta F1$	95% CI	p	Tokens
Planner	+0.083	[+0.038, +0.130]	0.006	−31%	−0.047	[−0.108, +0.013]	0.088	−21%
Backtracking	−0.013	[−0.038, +0.010]	0.727	−7%	0.000	[−0.031, +0.030]	1.000	−18%
Verification	+0.004	[−0.006, +0.018]	1.000	−2%	−0.009	[−0.024, +0.000]	1.000	−6%
Model scoring	−0.016	[−0.060, +0.026]	0.664	−25%	−0.008	[−0.046, +0.030]	0.481	−31%

6. Component-Level Attribution

Agentic systems are assemblies, and the literature reports them whole. This section takes AGR apart. Four conditions each remove exactly one component from the unmodified system, as Section 3.4 specifies, and re-run the identical questions: no planner, no backtracking, no claim verification, and embedding-only scoring ($\alpha = 1$, disabling the model term in the edge scorer). Each condition is compared against the reference on the same half-split, drawn as half of every hop stratum rounded down, which gives $n = 200$ on WebQSP and $n = 198$ on ComplexWebQuestions, with McNemar’s test on the paired per-question outcomes. The reference row is the unmodified system restricted to the same questions, read from the main-run records rather than re-run, so it is provably the system that produced Table 3. Each condition was run once, and the components were removed one at a time; the consequences of both choices are stated in Section 6.3.

Throughout this section a delta is ablated minus reference, so a positive number means removing the component *improved* the metric. Table 8 gives all four. The p -values are reported as computed. A correction for multiple comparisons was not among the pre-specified decisions, and applying one after seeing the results would be a choice made on the results, so the four tests are read as descriptive rather than as a family-wise claim.

6.1. When Planning Hurts

One component produced a detectable effect, and it produced it in the wrong direction. **Removing the planner improves WebQSP F1 by 0.083 ($p = 0.006$) while cutting**

token use by 31%. Twenty-one questions were answered correctly only without the planner, against six correct only with it. The disagreement is not marginal, and the cheaper system is the more accurate one.

The mechanism is over-decomposition. WebQSP is predominantly a single-hop benchmark: 256 of its 400 questions need one hop. Asked to decompose a question that has no compositional structure, the planner manufactures sub-objectives that are not really separable. The explorer then scores candidate edges against a fragment of the question rather than against the question, and it discards exactly the context that would have identified the right relation. The planner prompt carries a single-hop exemplar precisely to show that not decomposing is a valid output, and that exemplar is evidently not enough. The per-stratum breakdown locates the damage. On the WebQSP half-split, removing the planner raises one-hop Hits@1 from 0.75 to 0.85 and two-hop Hits@1 from 0.84 to 0.89, so decomposition hurts simple and two-hop questions alike on that benchmark.

On ComplexWebQuestions the direction reverses, as the mechanism predicts. That benchmark is genuinely compositional, only 137 of 400 questions are single-hop, and removing the planner costs 0.047 F1. **That arm does not reach significance** ($p = 0.088$), and we report it as a direction rather than an effect. We do not describe it as marginally significant, because at this sample size that phrase would assert more than the data carries. The strata show a coherent mechanism rather than noise. Without the planner, one-hop Hits@1 *improves* from 0.49 to 0.54, two-hop collapses from 0.48 to 0.34, and three-hop-plus moves from 0.62 to 0.54. That is exactly the pattern in which decomposition helps genuine composition and hurts questions that do not need it, and it is the pattern Table 5 shows from the other side, where the agentic baseline beats AGR only on the single-hop questions it finishes.

Pooling the two datasets would average a significant improvement against a non-significant harm and report approximately nothing. The asymmetry *is* the result. Decomposition is not a general-purpose benefit. It is a mechanism whose value depends on whether the question has compositional structure to find. The actionable consequence follows directly, and it is the recommendation we would most like to see adopted: **gate decomposition on question structure** rather than applying it unconditionally, predicting the number of hops

from the question and invoking the planner only above a threshold. On these benchmarks that policy would capture the WebQSP improvement and the ComplexWebQuestions capability at once.

6.2. Three Components with No Detected Effect

The remaining three conditions moved nothing detectable, and we report them at the same length as the one that did. A field that reports only what moved cannot attribute anything.

Backtracking. Removing it changes WebQSP F1 by -0.013 ($p = 0.727$) and ComplexWebQuestions F1 by 0.000 ($p = 1.000$), while saving 7% and 18% of tokens. The mechanism fires rarely. The reference condition records 38 and 108 backtracks over the 398 paired questions, and on those same runs the meter refuses a further attempt on 28 of them, 7.0%. A component this seldom exercised cannot show a large effect. One implementation detail bounds the null further, and we record it rather than smooth it over. The backtracker bans the edges of the most recent explorer pass but restores the highest-scoring snapshot, which is often older, so everything expanded in between stays unbanned (Section 3.1). Over the main runs, 53 explorer passes re-expanded an anchor-and-relation set the same question had already expanded, and at least 65 of the 262 backtracks restored a depth other than the most recent snapshot’s. The null therefore describes backtracking as implemented, not backtracking as specified. The accurate summary is that the search rarely needs to undo a decision, not that undoing is worthless.

Model scoring. Setting $\alpha = 1$ removes the language model from edge selection entirely, leaving pure embedding similarity. F1 falls by 0.016 and 0.008, neither significant, while tokens drop 25% and 31%. The model term is the single most expensive part of the navigation loop, and this experiment cannot establish that it earns its cost. The development-set sweep that froze α points the same way at a smaller scale: at $\tau = 0.20$ it scored 61, 62, 64, and 59 hits of 80 at $\alpha = 0.3, 0.5, 0.7$, and 1.0 , so dropping the model term cost five hits there, before the entity-filter correction that preceded the test runs. One confound is recorded rather than smoothed over. At $\alpha = 1$ the scorer takes an early return that leaves the backtracking trigger’s signal maximum computed over a smaller candidate set, so this condition’s elevated backtrack count is not attributable to the scoring change

alone.

Claim verification. Removing the checking routes of the layer AGR was designed around, while leaving the drafting call unchanged, changes WebQSP F1 by +0.004 and ComplexWebQuestions F1 by -0.009 , $p = 1.000$ on both. Across the half-splits — 200 questions on WebQSP and 198 on ComplexWebQuestions — the two conditions disagreed on exactly one question each. **Claim verification does not measurably change how often AGR is right.**

The result is expected rather than disappointing, for the reason Section 3.2 gave before any of these numbers appeared. On most questions the layer is a pass-through certifying a draft that was already grounded. On test it asks for a repair on 52 of 800 questions, exploration actually resumes on 28, and 13 end grounded that did not start so. A mechanism that alters the output on a few per cent of questions cannot move an aggregate accuracy metric. An ablation is the right instrument for establishing that, rather than assuming it either way.

What the layer delivers is therefore not an accuracy gain but the output contract: an answer paired with the traversed triples that support it, and an entity list that is a deterministic function of the verdicts rather than a second generation. That is an auditability claim, not an accuracy claim, and Section 8.1 states how far it reaches: which route accepts most claims is not recorded, and the supporting triples are counted rather than kept. Section 5.6 has already shown that the layer cannot claim credit for groundedness.

6.3. Statistical Power

These are nulls at low power, not demonstrations of absence, and the arithmetic is worth stating exactly. McNemar’s exact test sees only the *discordant* pairs, the questions on which the two conditions disagree, and the null conditions produced very few of them.

Removing claim verification changed the outcome on exactly one question per dataset. With a single discordant pair no split whatsoever can reach significance. So that row is not a test that failed to reject. It is the two conditions agreeing on 396 of the 398 paired questions, which is a statement about how rarely the layer alters an answer rather than about statistical power. Backtracking produced 8 and 11 discordant pairs, and model scoring 21 and 18. At those counts the smallest imbalance the exact test could have called significant is 8 and 9

questions for backtracking, meaning all eight would have had to fall the same way, and 11 and 10 for model scoring, which is between 4.0 and 5.5 points of accuracy. Differences below that were undetectable at any power, whatever their true size.

For calibration: detecting a true 2:1 discordance ratio at 80% power requires roughly 72 discordant pairs, and the largest of these conditions produced 21. At that count the smallest ratio detectable at 80% power is nearer 4:1. A true 2:1 effect would have been caught about a quarter of the time. **Three of four components show no effect at this power**; none is shown to have no effect.

One more count bounds what any single removal could have done. The union of the four discordant sets holds 42 of the 200 WebQSP questions and 55 of the 198 on ComplexWebQuestions. On the remaining 158 and 143, no condition changed the outcome. The intervals in Table 8 say the same thing in F1: the planner’s WebQSP interval is the only one of the eight that does not contain zero.

Two features of the design bound these numbers further. Each condition is one run against a hosted model whose temperature-zero decoding is greedy rather than deterministic, so a delta of a few hundredths sits inside the run-to-run variance that Section 8.2 reports, and none of the three nulls was replicated. And the components were removed one at a time. Three nulls in one-at-a-time removal are consistent with the three components being individually dispensable and jointly necessary, and equally with a system that has none of them matching this one at a fraction of the tokens. The condition that separates those readings, all three removed together, was not run. Section 8.4 names it, with the replication, as the next experiment.

7. Error Analysis

After the label-defect exclusions of Section 7.2, AGR’s residual non-hits on the two samples number 262: 86 on WebQSP and 176 on ComplexWebQuestions. Six of them, one and five, are the surface-form near-misses of Section 4.3, answers that strict matching rejects and aggressive normalisation accepts; they are scoring artefacts rather than reasoning failures and were set aside. Every one of the remaining 256, 85 and 171, was read and labelled by hand against its full execution trace. The counts below are therefore counts over

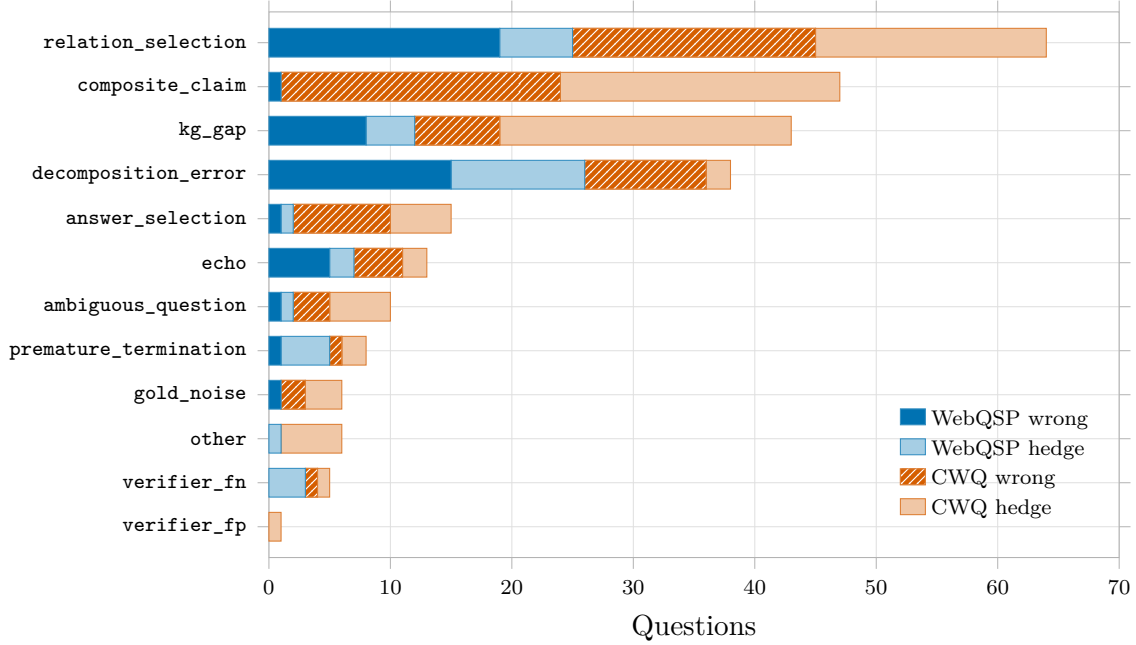


Figure 5: Hand-read census of the 256 residual failures read, by category, with wrong answers and hedges kept separate. Relation selection and composite claims dominate. The two categories this section discusses are not the largest, but they are the two whose lessons transfer beyond this system.

a population rather than estimates from a sample, and no category is extrapolated. The reading protocol was fixed before it began. For each trace, find where the trajectory *first* left the correct path, and label that point rather than the last visible symptom. Wrong answers and hedges are kept separate throughout, because the two are not the same kind of failure. A wrong answer is a reasoning error. A hedge is usually a coverage gap that never produced a committal claim. Figure 5 gives the census.

The bulk of the distribution is unsurprising. Choosing the wrong relation to follow is the base rate of graph navigation, and it is the largest category overall. Composite claims, in which an unverifiable component drags down a verifiable one, are nearly absent on WebQSP and the largest category on ComplexWebQuestions, because that benchmark’s template-generated questions carry conjunctions and constraints that WebQSP’s search-log questions rarely do. The same construction explains the third-largest category. Questions whose answer is a date, a number, or a superlative fall into the class Section 4.1 named as unanswerable in this environment, and on ComplexWebQuestions the system hedges on

them more than three times as often as it asserts something wrong, 24 against 7. That is the behaviour the drafting prompt’s grounding provisions were written to produce, visible in the one place where the environment guarantees the system cannot succeed. Two smaller categories are reported in full instead, because they generalise past this architecture.

7.1. *The Echo Attractor*

The most instructive failure mode is one that every grounding check in this paper passes. The system resolves a real entity, reached through a real edge, and states a fact that is genuinely true. It is not what was asked. We name this the **echo attractor**: a true, well-grounded entity sitting one hop from the answer in the graph’s structure.

It takes two shapes. *Topic confusion* picks a neighbour in a tight naming cluster. Asked who plays Lex Luthor on *Smallville*, the system answers that John Glover plays Lionel Luthor, a different character in the same fictional family, with three further Luthor variants surfacing in the trace before the run gives up. *Granularity* picks the right entity family at the wrong level. Asked for a country, the system answers “Kingdom of Denmark” where the gold answer is “Denmark”, a genuinely distinct node, the formal sovereign-state entity rather than the common-name one. A reader may prefer to call the second shape a label-granularity problem rather than a system error. We count it against the system because the two nodes are distinct in the graph and the question’s gold names one of them, and the label-defect pass of Section 7.2 is where the other reading was tested, question by question.

What makes this a mechanism rather than a residue category is that nothing went wrong in the parts of the system built to prevent things going wrong. The right edge exists, is retrieved, and is walked. The entity returned is verifiably present in the environment. Every structural check certifies it. The failure is entirely in *which* true fact gets surfaced, which is architecturally distinct both from following a wrong edge and from the fact being absent. A claim-level check against traversed adjacency cannot catch it by construction. The claim is true.

The consequence for evaluation methodology is larger than the consequence for this system. Systems that share a graph and a backbone converge on the *same* wrong entity for the *same* questions, and Section 7.2 measures how often. Any protocol that treats systems as independent observers therefore cannot see it. Consensus filtering,

ensembling, and majority-vote gold repair all take agreement as evidence of correctness, and here agreement is evidence of a shared bias instead. Such a protocol will confidently ratify the wrong answer in exactly the cases it was introduced to catch.

The echo attractor accounts for 13 of the 256 failures read. The rate is small and the sample does not support a precise one. The contribution is the named mechanism and its methodological consequence, not its frequency at this n .

7.2. Benchmark Label Defects

The census required a second pass, because a failure against a wrong gold label is not a failure of the system. A consensus pre-pass flagged every question where at least three of the five systems agreed on an answer the benchmark scored wrong. Each flagged question was then adjudicated individually against the graph and the question text.

The two counts must be kept apart. The pre-pass **flagged** 58 WebQSP and 47 ComplexWebQuestions questions. Adjudication **confirmed** a genuine label defect in 22 and 19 of them, which is 5.5% and 4.8% of each sample. Confirmed defects split into two kinds: gold answers that are simply wrong (12 and 17), and questions whose phrasing admits several defensible answers of which the benchmark lists one (10 and 2). All 41 were removed before the census began, as benchmark artifacts rather than system failures.

The hand-read then found 16 more that the pre-pass had missed — 3 on WebQSP and 13 on ComplexWebQuestions — and filed them under its own gold-noise and ambiguous-question labels. So 57 **distinct questions** across the two benchmarks turned out to need correcting. That the census recovered defects a dedicated consensus pass had missed is itself an argument for reading failures individually rather than screening them in aggregate.

The gap between flagged and confirmed is not adjudication noise. The pass flagged 105 questions and adjudication confirmed 41. So roughly three-fifths of what it flagged was not a bad label at all but several systems converging on the same wrong entity. Table 9 reads those 64 cleared questions row by row, where a row is one consensus answer and a question can carry more than one. The triage filed 52 of the 85 cleared rows as the echo mechanism of Section 7.1 and the rest as relation confusion, an ignored constraint, or the environment gap, so the gap between flagged and confirmed is mostly, not entirely, the echo attractor. The table also shows that the agreeing systems are not independent observers for a second reason

Table 9: The 64 flagged questions that adjudication cleared, read as consensus rows (one row per consensus answer). The triage family is the pre-pass’s own label for why the systems agreed on a wrong entity. The lower rows count which systems were among those agreeing.

	WebQSP	CWQ
Consensus rows cleared as not a label defect	54	31
filed as echo attractor	36	16
filed as relation confusion	16	3
filed as constraint ignored	0	12
filed as environment gap	2	0
Rows on which the no-retrieval control agrees	15	25
Rows on which both navigators agree	26	14
Echo rows on which the no-retrieval control agrees	13	11
Echo rows on which both navigators agree	20	9

besides the graph: they share one backbone and one resolver. On ComplexWebQuestions the no-retrieval control, which sees no graph at all, is among the agreeing systems on 25 of the 31 cleared rows and on 11 of the 16 echo rows, so there the convergence is at least as much a property of what the backbone believes as of the graph’s neighbourhood structure. On WebQSP the control joins 15 of 54 rows and 13 of 36 echo rows, and the two navigators agree with each other on 20 of the 36. Of the 13 rows on which all five systems agreed, 9 were cleared as not a defect. The methodological consequence does not depend on which cause dominates. A protocol that takes agreement as evidence of correctness ratifies the shared answer either way, and had we accepted the pre-pass without per-item adjudication, we would have “corrected” the benchmark toward the systems’ shared bias and reported an inflated accuracy for every system, including our own.

The accuracy tables are not rescored. Section 5 reports every system exactly as measured over the full samples, for two reasons. Rescoring against labels we adjudicated ourselves would substitute one contestable ground truth for another. And the defects cut both ways. A broken label can just as easily hand a system a hit as deny it one, and 10 of the 22 WebQSP exclusions and 4 of the 19 on ComplexWebQuestions are questions AGR scored as hits against a label now judged defective.

That symmetry also bounds how much the exclusions could matter, which is worth

stating because the defect rate on its own invites a much larger inference. Recomputing every cell of Table 3 with the adjudicated questions removed moves each system by between +0.001 and +0.020 Hits@1 and leaves the ranking on both datasets unchanged. That shift is an order of magnitude below the 5.5% and 4.8% defect rates, because removing a question the system got right subtracts from the numerator as well as the denominator. The exclusions therefore do not threaten any comparison this paper draws. What they do establish is that a non-trivial share of both benchmarks is mislabelled, which is a fact about the benchmarks rather than a correction to be applied to them. The adjudication was made by one author with the consensus signal visible, and Section 8.3 carries that as the limitation it is.

8. Discussion and Limitations

The limits of this study bound its claims, and we state all of them rather than the flattering subset. The first group bounds the instrument, the second bounds the comparisons, and the third bounds the measurements.

8.1. *What Bounds the Instrument*

Wrongful acceptance is unmeasured, and it is the most serious gap in this work. The verifier logs the claims it rejects and not the ones it accepts. So a claim certified on an edge unrelated to the relation it asserts leaves no trace. The exposure is real rather than hypothetical, because both structural routes ignore the relation and the direction. Any edge between two entities certifies any claimed relation between them, so a claim that X is Y’s mother is accepted on an edge recording that X is Y’s child. We therefore report one polarity at census scale and the other anecdotally, and that asymmetry is a property of the logging decision rather than of the verifier. There is no way, from the committed record, to know how many wrongful acceptances sit inside the 761 questions that ended grounded.

The exposure is best stated in the claim unit, where its scale is visible. On test the verifier decomposed 2,110 claims and accepted 2,008 of them. The graph-wide adjacency probe accounts for 39 of those acceptances. The rest were made either by traversed adjacency, which is equally relation-blind, or by the entailment check, which is not, and the record does not say which. So the share of accepted claims certified without any test of the asserted relation lies somewhere in [39, 2,008]. That interval is two orders of magnitude wide, and we

report it as one rather than choose a point inside it. The same gap means the layer has no measured precision or recall as a classifier of claims, which is the first number a verification component should carry.

The output contract cannot be audited from the run records. AGR emits an answer paired with the traversed triples supporting it, and the pairing does hold inside the run: the answerer receives the verifier’s triple list and emits against it. But the logger writes the *count* of supporting triples and discards the list, so no committed artifact in this work contains a single supporting triple. On test, 12 of AGR’s 367 answered WebQSP questions and 9 of its 308 on ComplexWebQuestions were emitted with a count of zero, which is the contract’s own measure of how often it delivers nothing. A reader can confirm that the other answers came from a system that tracked its evidence. They cannot inspect that evidence. Persisting accepted claims with their route and their matching triples is one logging change, and it would repair this limitation and the previous one together.

Structural grounding is not answer adequacy. Both navigating systems assert zero ungrounded entities, and under the definition used that is close to guaranteed for any system that copies entities out of tool results. It says their answers are made of the graph’s own material. It does not say they answer the question. The echo attractor of Section 7.1 is the counter-example: a wrong answer that passes every check this paper applies. The verification layer offers no protection against it.

8.2. *What Bounds the Comparisons*

Sample size. Every result rests on 400 questions per dataset rather than the full splits, giving 95% bootstrap intervals of roughly ± 4 to ± 5 points on Hits@1. The main accuracy margins clear that width: 9.5 points over the agentic baseline on WebQSP and 8.7 on ComplexWebQuestions. So does the planner effect, which is a paired test on per-question outcomes rather than a comparison of two intervals. The groundedness contrast, at zero against 22.1%, is not close. What the sample does not support is fine distinctions. The semantic-tier gaps of Section 5.6, the hedge-rate difference on ComplexWebQuestions, and the last step of the rising hop-depth curve are within noise, and we report them as such.

The margin is a statement about one budget. The 25-call cap was pre-specified, but it is a choice, and Section 5.3 shows that the entire margin over the agentic baseline lives

on the questions where that cap truncates it. The split that shows this conditions on the baseline’s own outcome, and within hop strata the clipped half is as hard as the finished half on WebQSP and harder on ComplexWebQuestions. A cap sweep would move the boundary, and the paper reports the clip rates so a reader can see how far, but it did not run the sweep.

An identical environment is not identical access to it. The agentic baseline truncates candidate sets at 40 relations and 20 neighbours against AGR’s 300 and 200. That cut binds on about a third of its expansions and effectively none of ours (Section 5.3). The comparison holds the graph constant and does not hold the view of it constant, which is why the baseline’s figures on the questions it finishes are a lower bound rather than an estimate. The agentic comparator is also one system. Plan-on-Graph, whose two mechanisms AGR’s planner and backtracker re-host (Section 2.2), was not run, so the planner result speaks to decomposition as implemented here and to Plan-on-Graph’s by analogy only.

The static graph baseline is radius-bounded. Its one-logical-hop expansion cannot contain a two-hop chain, so its decay across hop strata measures the radius as much as the paradigm, and Section 5.5 declines to offer it as evidence for either. Its unordered 100-edge fanout cap compounds this.

One backbone. All five systems run on `gpt-5.4-mini`, frozen and closed. This paper establishes how these architectures compare *on one backbone*, not how the comparison scales across model capacities.

Nondeterminism. Temperature-zero decoding against a hosted API is greedy, not deterministic, and measured trajectory stability is about 67%. Each system and each ablation condition was run once. No seeds were averaged, and a reader should not assume they were. The intervals and paired tests resample questions, so they bound the sampling variance of the question draw and nothing else; each system’s answer set enters them as fixed. Two things make a rank reversal from decoding noise alone unlikely: the defended margins are large relative to their intervals, and the paired tests are computed per question. But unlikely is the strongest statement available. The repeated matrix that would bound the variance was affordable at the observed cost and was not run, so this is a gap in the design rather than a budget constraint.

Ablation power. The four ablations run on half-splits of $n = 200$ and $n = 198$, and

the paired test sees only the questions on which two conditions disagree, as few as one for claim verification. Three of four components show *no effect detected at this power*, which is not the same as no effect. Section 6.3 gives the smallest difference each condition could have detected, between 4.0 and 5.5 accuracy points where a test was possible at all. The components were also removed one at a time, so the three nulls do not say whether a system with none of them would match this one; that condition was not run.

8.3. What Bounds the Measurements

The environment is friendlier than the knowledge base it stands in for. Per-question subgraphs with a 97%-plus answer-reachability ceiling are a more forgiving substrate than full Freebase. Between-system comparisons hold, because all five systems navigate the identical graph. Absolute accuracies do not transfer, and Section 5.4 shows the size of the gap to a published system that was also trained on the benchmarks.

Topic entities are given, not linked. Both graph-navigating systems seed from the datasets’ annotated topic entities. That isolates navigation from entity linking, which is the intent. But it means every reported accuracy assumes a linking step a deployed system would have to perform, and which would introduce errors of its own.

Entities are keyed by name, so homonyms merge. Distinct real-world entities sharing a surface form collapse to one node. That can manufacture a path between things not actually connected, letting a wrong answer verify as grounded, and it can equally destroy a distinction a question depends on. Neither direction is measured. Structural groundedness is therefore a claim about this graph’s node identity rather than about the world.

The measured accuracy has an unmeasured floor. An extraction defect converts correct reasoning into a wrong entity list on one specific question shape. On “list the professions” and “what did X do” questions the drafted text names every gold value and the scored entity list collapses to the sentence’s subject. Accuracy on that shape is a lower bound rather than an estimate.

The judge missed its own bar, and the adjudication was single-annotator. Agreement between the semantic-tier judge and a human annotator reached $\kappa = 0.6995$ against a pre-specified 0.70. That is a miss, by 0.0005. It is why those numbers are reported as corroborating rather than decisive, and why no conclusion rests on them alone. The judge

also runs on the same backbone as the systems it judges. Separately, the benchmark-defect exclusions and the failure census rest on single-annotator judgements made by an author with the consensus signal visible (Section 7). One development-set relabelling was also made after its outcome was known. It is flagged in place, and no aggregate was rescored.

Scope. English factoid questions over a static graph, with no temporal facts, no graph construction, and an offline prototype. The environment carries no typed literals and the tool API no ordering operator, so questions answered by a date, a quantity, or a superlative are out of reach by construction (Section 4.1). Wall-clock figures are reported as magnitudes with their cache caveat (Section 5.2), not as a production-readiness claim.

8.4. Outlook

The findings above fix the order of the next experiments, and each is cheap against the cost of the study it would complete. First, the agentic baseline at AGR’s candidate widths under a sweep of the call cap, with the budget split of Table 4 reported at each setting, which tests whether the affordability reading survives equal access to the graph. Second, replication: every system and every ablation condition run several times, so that a delta of a few hundredths of F1 can be read against the run-to-run variance rather than assumed to exceed it, together with the joint-removal condition that Section 6.3 names. Third, the full test splits, which quadruple the discordant pairs the paired tests see. Fourth, one logging change: persist each accepted claim with the route that accepted it and the triples that matched. That converts the two limitations opening this section into measurable quantities, and it makes the verifier’s precision and recall a number a labelled sample can produce.

The most actionable finding points forward rather than back. If decomposition helps only when a question has compositional structure, improving accuracy by 0.083 F1 where that structure is absent and costing 0.047 where it is present, then the planner should not run unconditionally. A hop-count predictor gating the planner would capture both arms of that asymmetry, and it can be evaluated with the ablation instrument used here. More generally, the result suggests that agentic components should be treated as conditionally rather than universally beneficial, and that the condition is worth measuring.

9. Conclusion

We built an agentic knowledge-graph question-answering system whose four components can be removed one at a time, ran it against four baselines on one frozen backbone under one call budget, and took it apart. The aggregate result is the one the field usually reports: the agentic system has the highest accuracy in every column, at roughly half the model calls of the agentic baseline. The rest of the paper is what that result turns out to be made of. The entire margin over the agentic baseline comes from the questions on which the shared budget truncates it; where the budget lets that baseline finish, it is ahead. The margin is a finding about affordability under a fixed spend, and it is bounded by a candidate-width confound we measured rather than assumed. Against the published figures of a system fine-tuned on these benchmarks, AGR trails on every number, and we report that too.

The result we consider most useful is the one that did not go our way. Of four components, three could not be shown to earn their cost at the power available, and we state the smallest effect each test could have caught. That includes the claim-verification layer the system was designed around, which changes the answer on one question per dataset. What it delivers is an output contract rather than an accuracy gain, and the contract is narrower than its description: one route of three attaches evidence, and the evidence is counted rather than kept. The fourth component was actively harmful on the shallower benchmark. Removing the planner improved accuracy and cut token use, because decomposing a question that has no compositional structure discards the context that would have answered it. Decomposition is conditionally beneficial, and the condition is measurable.

The shape of the residue matters as much as the margin. A system that traverses a real graph and copies its answers out of real triples can be made to assert nothing ungrounded. That problem is solved, and the navigation paradigm solves it, not any checking layer built on top. What remains is an answer that is grounded, connected, and true, and is still not the answer to the question that was asked. Systems sharing a graph and a backbone converge on that answer together, so a protocol that takes their agreement as evidence ratifies it. Fact verification against a knowledge graph turns out to be the easier half of the problem. Relevance verification is the half still open.

The attribution result is a claim about how this field measures rather than about this

architecture. A component adopted because the assembly containing it performed well may be contributing nothing, or less than nothing, and a margin reported as an architectural advantage may be a budget regime. The only way to know is to remove the component, vary the budget, and test. We would rather report three nulls, a harmful component, and a margin that lives in a cap than a system that works for reasons nobody has checked.

Appendix A. Prompt templates

Seven templates constitute the whole of AGR’s interface to the language model, and six serve the baselines: one for the parametric control, one shared by the two static retrieval baselines, and four belonging to the reimplemented agentic baseline. All thirteen are reproduced here verbatim from the source that sends them, in the order a question encounters them, by a script that reads the code (`scripts/build_paper_appendix.py` in the repository named in the data availability statement). No template carries a configuration constant, so one cached response serves every condition of a sweep; budgets are counters in code rather than instructions, and the evaluator’s remaining depth and backtracks are the only budget figures any prompt carries. Every call asks for JSON against a schema stated in the code beside the template.

Appendix A.1. Planner

`agr/planner.py`, constant `PLANNER_PROMPT`. Sent once per question unless the planner is removed. The first exemplar is the single-hop case that is not decomposed. Braces written `{like this}` are format fields filled at send time; doubled braces are literal.

```
You decompose questions into a minimal ordered chain of sub-objectives for navigating a
knowledge graph. Each sub-objective must be answerable by following one relation (
possibly through an event node). Use #N to reference the result of sub-objective N. Use
as FEW sub-objectives as possible.
```

```
For topic_mentions, extract ONLY specific named entities that appear literally in the
question (people, places, organizations, works, products). Do NOT extract generic type
words such as "celebrity", "university", "country", "senator", "airport", or "currency"
-- these describe what kind of answer is wanted, not where to start searching.
```

```
Example 1 (single hop -- do NOT decompose):
Question: "who is the president of France?"
{"sub_objectives": ["find the president of France"],
 "topic_mentions": ["France"]}
```

Example 2 (composition):

Question: "who directed the film that won Best Picture in 1998?"

```
{"sub_objectives": ["find the film that won Best Picture in 1998",  
                    "find the director of #1"],  
  "topic_mentions": ["Best Picture"]}
```

Example 3 (conjunction):

Question: "which countries border France and have Spanish as official language?"

```
{"sub_objectives": ["find countries that border France",  
                    "find which of #1 have Spanish as an official language"],  
  "topic_mentions": ["France", "Spanish"]}
```

Example 4 (superlative):

Question: "what is the largest city in the country where the Nile ends?"

```
{"sub_objectives": ["find the country where the Nile ends",  
                    "find the largest city in #1"],  
  "topic_mentions": ["Nile"]}
```

Question: "{question}"

Appendix A.2. Relation scorer

agr/scorer.py, constant SCORE_PROMPT. One batched call per explorer pass over the twenty candidates with the highest embedding score. Not sent when $\alpha = 1$. Braces written {like this} are format fields filled at send time; doubled braces are literal.

Sub-objective: {objective}

Candidate knowledge-graph edges (anchor entity --relation-->):
{candidates}

For EACH candidate, rate from 0.0 to 1.0 how likely following that edge from that anchor leads toward completing the sub-objective. 1.0 = directly answers it, 0.0 = irrelevant. Judge the RELATION MEANING in context of the anchor, not surface word overlap.

Appendix A.3. Evaluator

agr/nodes.py, constant EVAL_PROMPT. One call per explorer pass. The remaining depth and backtracks are substituted at send time; no other prompt carries a budget figure. Braces written {like this} are format fields filled at send time; doubled braces are literal.

Current sub-objective: {objective}

Recently retrieved facts:

{facts}

Remaining budget: {d_left} more expansions, {b_left} more backtracks.

Decide:

- "answer" if the facts complete the sub-objective (list satisfying entities in "resolved", exactly as named in the facts),

- "continue" if this direction is promising but incomplete,
- "backtrack" if this direction is a dead end.

Appendix A.4. Drafter and claim decomposer

agr/nodes.py, constant DRAFT_PROMPT. One call when the search ends. Produces the prose, the answer entities, and the atomic claims the verifier checks. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Facts retrieved from the knowledge graph:
{facts}
Entities identified as promising during exploration:
{candidates}

1. Draft an answer using ONLY the facts and candidates above. Use entity
   names exactly as written. If insufficient, say what could not be
   determined.
2. List the answer entities themselves in "answer_entities", copied EXACTLY
   as they are named in the facts above -- these are the entities your draft
   asserts as the answer, not every entity it mentions. If your draft is a
   hedge ("could not be determined"), "answer_entities" MUST be [].
   IMPORTANT: "answer_entities" must be specific entities of the kind the
   question asks for (a person for "who", a place for "where", etc.).
   Never restate the question or name entity types/categories as the
   answer. If you cannot name specific answer entities from the facts,
   your draft must be a hedge and "answer_entities" must be [].
3. Decompose your draft into atomic factual claims, each as
   {"h": entity name, "r": short relation phrase, "t": entity name}}.
   Only include claims your draft actually asserts. A hedged draft has zero
   claims.
```

Appendix A.5. Entailment check

agr/nodes.py, constant ENTAIL_PROMPT. One batched call over the claims neither structural route settled. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Retrieved knowledge-graph triples:
{facts}

For each claim below, answer whether it is directly supported by the triples
above (true) or not (false). Unsupported includes: plausible but absent,
contradicted, or only inferable via outside knowledge.
Claims:
{claims}
```

Appendix A.6. Rewriter

agr/nodes.py, constant REWRITE_PROMPT. One call on the give-up path only. The entity list it is given is already filtered; its own opinion about entities is discarded. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Draft answer: {draft}
The following claims in the draft are NOT supported by the knowledge graph:
{unsupported}
The following entities remain verified and MUST be kept in the answer,
named exactly as written: {kept}

Rewrite the draft: keep the verified entities and their supported claims,
and remove or explicitly hedge the unsupported claims. If the kept list is
empty, state that the answer could not be verified against the knowledge
graph. Respond with prose only.
```

Appendix A.7. Answerer (verifier skipped)

agr/nodes.py, constant ANSWER_PROMPT. Reached only when the drafting call itself failed; no committed run took this path. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Facts retrieved from the knowledge graph:
{facts}

Entities identified as promising during exploration:
{candidates}

Answer using ONLY these facts and candidates. Give the answer entity name(s)
exactly as they appear above. If the information is insufficient, state what
could not be determined.
Also list in "answer_entities" the entities your answer
asserts (exactly as named above); [] if it asserts nothing.
```

Appendix A.8. No-retrieval control

agr/baselines/noretrieval.py, constant PROMPT. The single call of the parametric control. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Answer this question with the name(s) of the answer entity or
entities. Be concise. If you do not know, say so and return [].

Question: {question}
```

Appendix A.9. Vector-RAG and static GraphRAG

agr/baselines/vectorrag.py, constant PROMPT. Shared by the two static retrieval baselines, which differ only in what they retrieve. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Facts (retrieved from a knowledge graph):
{facts}

Question: {question}
Answer using ONLY the facts above. Name entities exactly as written in the
facts. If the facts are insufficient, say so and return [].
```

Appendix A.10. Think-on-Graph: relation pruning

agr/baselines/tog.py, constant REL_PRUNE. One call per frontier entity per depth, over the first 40 relations returned. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Current entity: {entity}
Available relations (relation, direction, fanout):
{relations}
Select up to {w} relations most likely to lead to the answer.
Return [{"relations": [{"rel": "...", "dir": "in|out"}]}].
```

Appendix A.11. Think-on-Graph: entity pruning

agr/baselines/tog.py, constant ENT_PRUNE. One call per selected relation per depth, over the first 20 neighbours returned. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Candidate entities reached (via {entity} --{rel}-->):
{entities}
Select up to {w} entities most likely on the path to the answer.
Return [{"entities": ["..."]}].
```

Appendix A.12. Think-on-Graph: sufficiency

agr/baselines/tog.py, constant SUFFICIENT. One call per depth, on the last sixty gathered triples. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Knowledge triples gathered so far:
{triples}
Can the question be answered from these triples alone?
Return {"sufficient": true|false}.
```

Appendix A.13. Think-on-Graph: answer

agr/baselines/tog.py, constant ANSWER. One call when the search stops, whether it finished or hit the cap. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Knowledge triples:
{triples}
Answer using ONLY these triples; name entities exactly as written.
If insufficient, say so and return [].
```

CRedit authorship contribution statement

Md. Sakif Khan: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Sadia Sharmin:** Conceptualization, Methodology, Supervision, Project administration, Writing – review & editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare no competing financial interests or personal relationships that could have influenced the work reported here.

Data availability

Code, prompts, per-question run records, and the analysis scripts that produce every figure and table in this paper are available at <https://github.com/sakif-khan/agent-ic-graph-reasoning> under the MIT licence. The question sets are the public WebQSP [9] and ComplexWebQuestions [10] releases, and both they and the Freebase-derived [3] subgraphs arrive through the public Reasoning-on-Graphs distribution [7] (`rmanluo/RoG-webqsp` and `rmanluo/RoG-cwq`), used unmodified. The knowledge environment is rebuilt from that distribution by the scripts in the repository rather than redistributed.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to draft and revise the prose of this manuscript from the completed thesis on which it is based, and to write the scripts that check the values quoted in the text against the study’s recorded results. The study design, the experiments, the analysis, and the findings reported here are the authors’ own and predate the preparation of this manuscript. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

- [1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, *ACM Transactions on Information Systems* 43 (2) (2025) 1–55.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 9459–9474.

- [3] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, J. Taylor, Freebase: A collaboratively created graph database for structuring human knowledge, in: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, 2008, pp. 1247–1250.
- [4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, J. Larson, From local to global: A graph RAG approach to query-focused summarization, arXiv preprint arXiv:2404.16130 (2024).
- [5] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, J. Guo, Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2307.07697.
- [6] L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, H. Xiong, Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs, in: Advances in Neural Information Processing Systems, Vol. 37, 2024, arXiv:2410.23875.
- [7] L. Luo, Y.-F. Li, G. Haffari, S. Pan, Reasoning on graphs: Faithful and interpretable large language model reasoning, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2310.01061.
- [8] H. Luo, H. E, Y. Guo, Q. Lin, X. Wu, X. Mu, W. Liu, M. Song, Y. Zhu, L. A. Tuan, KBQA-o1: Agentic knowledge base question answering with monte carlo tree search, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025, arXiv:2501.18922.
- [9] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, J. Suh, The value of semantic parse labeling for knowledge base question answering, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), 2016, pp. 201–206.
- [10] A. Talmor, J. Berant, The web as a knowledge-base for answering complex questions, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, pp. 641–651.

- [11] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, J. X. Yu, A survey of graph meets large language model: Progress and future directions, arXiv preprint arXiv:2311.12399 (2023).
- [12] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, Y. Su, HippoRAG: Neurobiologically inspired long-term memory for large language models, in: *Advances in Neural Information Processing Systems*, Vol. 37, 2024, pp. 59532–59569.
- [13] D. Li, Y. Niu, Y. Ai, X. Zou, B. Qi, J. Liu, T-GRAG: A dynamic GraphRAG framework for resolving temporal conflicts and redundancy in knowledge retrieval, in: *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025, pp. 11880–11889.
- [14] C. Mavromatis, G. Karypis, GNN-RAG: Graph neural retrieval for efficient large language model reasoning on knowledge graphs, in: *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 16682–16699, arXiv:2405.20139.
- [15] M. Li, S. Miao, P. Li, Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation, in: *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.20724.
- [16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: *International Conference on Learning Representations (ICLR)*, 2023.
- [17] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: *Advances in Neural Information Processing Systems*, Vol. 36, 2023.
- [18] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, in: *Advances in Neural Information Processing Systems*, Vol. 36, 2023.
- [19] S. Ma, C. Xu, X. Jiang, M. Li, H. Qu, C. Yang, J. Mao, J. Guo, Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval aug-

- mented generation, in: International Conference on Learning Representations (ICLR), 2025, arXiv:2407.10805.
- [20] L. Luo, Z. Zhao, G. Haffari, Y.-F. Li, C. Gong, S. Pan, Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models, in: International Conference on Machine Learning (ICML), 2025, arXiv:2410.13080.
 - [21] J. Jiang, K. Zhou, W. X. Zhao, Y. Song, C. Zhu, H. Zhu, J.-R. Wen, KG-Agent: An efficient autonomous agent framework for complex reasoning over knowledge graph, arXiv preprint arXiv:2402.11163 (2024).
 - [22] Y. Xu, S. He, J. Chen, Z. Wang, Y. Song, H. Tong, K. Liu, J. Zhao, Generate-on-graph: Treat LLM as both agent and KG for incomplete knowledge graph question answering, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, arXiv:2404.14741.
 - [23] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, J.-R. Wen, StructGPT: A general framework for large language models to reason over structured data, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, arXiv:2305.09645.
 - [24] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: A roadmap, IEEE Transactions on Knowledge and Data Engineering 36 (7) (2024) 3580–3599.
 - [25] C. Ma, Y. Chen, T. Wu, A. Khan, H. Wang, Unifying large language models and knowledge graphs for question answering: Recent advances and opportunities, in: Proceedings of the 28th International Conference on Extending Database Technology (EDBT), 2025, pp. 1174–1177.
 - [26] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, J. Weston, Chain-of-verification reduces hallucination in large language models, in: Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 3563–3578.

- [27] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: Proceedings of the 41st International Conference on Machine Learning, 2024.
- [28] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, D. Zhou, Large language models cannot self-correct reasoning yet, in: International Conference on Learning Representations (ICLR), 2024.
- [29] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-RAG: Learning to retrieve, generate, and critique through self-reflection, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2310.11511.
- [30] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling, Corrective retrieval augmented generation, arXiv preprint arXiv:2401.15884 (2024).
- [31] Y. Hu, W. Xuan, Q. Zhou, Z. Li, Y. Li, J. Hu, F. Fang, A self-correcting agentic graph RAG for clinical decision support in hepatology, *Frontiers in Medicine* 12 (2025) 1716327.
- [32] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, FActScore: Fine-grained atomic evaluation of factual precision in long form text generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 12076–12100.
- [33] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using siamese BERT-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, 2019, pp. 3982–3992.